

Arquitectura de Computadoras

UART

Trabajo Práctico N° 2

Diseño de una interfaz **UART** mediante **FSM** y **FIFO** en **SystemVerilog**, integrada con la **ALU** y la placa **Basys 3**.

Institución

Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y
Naturales

Repositorio

[github.com/FrancoPerotti/
computer-architecture-tps](https://github.com/FrancoPerotti/computer-architecture-tps)

Integrantes

Arnaudo, Federico Andrés

Docente

Martín Pereyra

Carrera

Ingeniería en Computación

Perotti, Franco José

Índice

1	Introducción	4
2	Objetivos y alcance	5
2.1	Objetivo general	5
2.2	Objetivos específicos	5
3	Arquitectura general	6
3.1	Separación de responsabilidades	6
3.1.1	Integración y acondicionamiento	7
3.1.2	Camino de recepción	8
3.1.3	Interpretación y cálculo	9
3.1.4	Camino de transmisión y salidas	10
3.2	Flujo de operación	11
3.3	Decisiones de diseño	11
3.4	Organización del proyecto	12
4	Núcleo UART y máquinas de estado	14
4.1	Contrato del enlace y formato UART 8N1	14
4.2	Generación del tick de muestreo	14
4.3	Protocolo de aplicación	15
4.4	Recuperación y casos inválidos	16
4.5	Estructura y codificación de las máquinas de estado	17
4.6	Receptor UART	18
4.6.1	Validación del bit de start	19
4.6.2	Muestreo y orden de bits	19
4.6.3	Estado de recuperación	19
4.7	Transmisor UART	19
4.8	Buffers FIFO	20
4.9	Controlador de transacciones	21
4.10	Recuperación de estados ilegales	22
5	Integración con la ALU y la Basys 3	23
5.1	Recorrido de una transacción	23
5.2	Resultado UART e indicadores	24
5.3	Coordinación y rendimiento	24
5.4	Interfaz gráfica de la computadora	25
5.5	Integración con la Basys 3	26
6	Verificación funcional	28
6.1	Criterio de validación	28
6.2	Bloques elementales: ALU, temporización, FIFO y UART	28
6.2.1	Criterio y estímulos	29
6.2.2	Resultado observado	29
6.2.3	Interpretación	31
6.3	Subsistemas: núcleo UART y controlador	31
6.3.1	Criterio y estímulos	32
6.3.2	Resultado observado	32

6.3.3	Interpretación	33
6.4	Verificación extremo a extremo	33
6.4.1	Criterio y estímulos	33
6.4.2	Resultado observado	34
6.4.3	Interpretación	35
7	Síntesis e implementación	36
7.1	Flujo sobre el dispositivo objetivo	36
7.2	Elaboración y comprobación del RTL	37
7.3	Del esquema RTL a los recursos físicos	42
7.4	Utilización de recursos	47
8	Timing y limitaciones	49
8.1	Qué se comprueba	49
8.2	Resultado posterior al ruteo	50
8.2.1	Márgenes temporales	50
8.2.2	Cruces asíncronos	50
8.2.3	DRC y metodología	51
9	Validación física	54
9.1	Objetivo y montaje	54
9.2	Programación y conexión	54
9.3	Pruebas realizadas	55
9.3.1	Casos dirigidos	55
9.3.2	Casos pseudoaleatorios	55
9.3.3	Correspondencia entre UART y LED	56
9.4	Resultados y criterio de aceptación	59
9.5	Interpretación	59
10	Conclusiones	60
	Referencias	61

Sección 1

Introducción

El Trabajo Práctico N° 1 permitió construir una unidad aritmético-lógica de ocho bits y separar el cálculo combinacional de la interfaz física de la **Basys 3**. Los operandos y el código de operación se cargaban entonces con switches y pulsadores. El resultado y las banderas se observaban en los LED. Esta organización permitió conservar la **ALU** y modificar por completo la forma de suministrarle datos.

En este segundo trabajo la computadora envía dos operandos y un código de operación por el conversor USB–UART de la placa, y recibe como respuesta el resultado. La transformación parece, en una primera mirada, un reemplazo de pines. Sin embargo, el enlace UART transmite un bit por vez, no comparte una señal de reloj con la FPGA y puede detenerse a mitad de una operación. El circuito ya no solo debe calcular, ahora debe recordar qué recibió, medir intervalos, administrar capacidad y regresar al estado inicial del protocolo cuando la entrada es inválida.

Las máquinas de estado finitas organizan ese comportamiento temporal. Una máquina valida y reconstruye los bytes de entrada; otra produce las tramas de salida; una tercera interpreta la secuencia A–B–opcode. Entre productor y consumidor se interponen buffers **FIFO**. En este caso, su función no es aumentar la velocidad de la UART, sino impedir que diferencias breves de ritmo se conviertan en pérdidas o sobrescrituras silenciosas de información.

Sección 2

Objetivos y alcance

2.1 Objetivo general

El objetivo de este trabajo es diseñar, describir y verificar un sistema digital capaz de recibir dos operandos y un código de operación mediante UART, ejecutar sobre ellos la ALU desarrollada en el TP1 y transmitir el resultado por el mismo enlace. Además, el sistema debe ser sintetizable, reproducible y capaz de volver al estado inicial del protocolo después de recibir una trama incompleta o inválida.

2.2 Objetivos específicos

- Mantener el contrato funcional de la ALU del TP1, incluidas las operaciones ADD, SUB, AND, OR, XOR, NOR, SRA y SRL.
- Implementar una UART 8N1 a 19 200 baud con sobremuestreo 16x.
- Describir FSM separadas para recepción, transmisión y control de la transacción con recuperación segura.
- Desacoplar comunicación y procesamiento mediante FIFOs.
- Definir un protocolo binario sobre el orden de los bytes y el tratamiento de errores.
- Verificar cada bloque de manera aislada y comprobar después el recorrido completo desde el pin RX hasta el pin TX.
- Proveer una interfaz gráfica en Python que permita operar la ALU, ejecutar suites reproducibles y registrar la validación física.

Sección 3

Arquitectura general

3.1 Separación de responsabilidades

El sistema completo debe resolver problemas de naturaleza diferente. Por un lado, debe observar una entrada asíncrona y reconstruir una trama UART bit a bit; por otro, debe conservar bytes mientras los bloques avanzan a ritmos distintos, reconocer el orden A–B–opcode, ejecutar la operación y volver a serializar el resultado. Reunir esas tareas en un único módulo mezclaría tiempos de bit, reglas del protocolo y cálculo aritmético. Por eso el diseño elegido distribuye el trabajo entre módulos con interfaces explícitas.

La figura 3.1 presenta la arquitectura general y el recorrido completo de una operación.

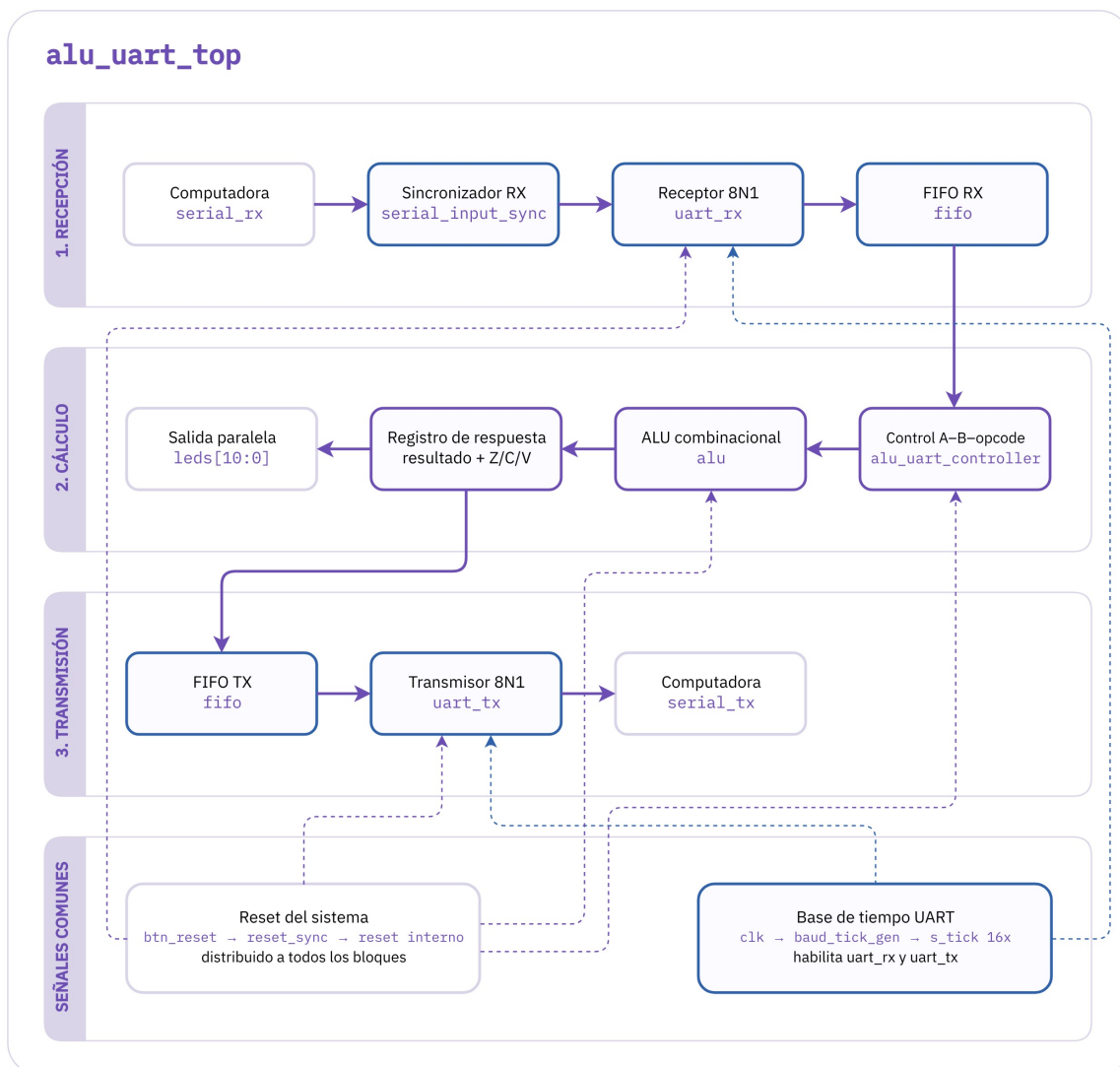


Figura 3.1: Recorrido de los datos y separación funcional de la arquitectura.

3.1.1 Integración y acondicionamiento

`alu_uart_top` es el módulo superior y representa el circuito conectado a la placa. Expone únicamente el reloj de 100 MHz, el pulsador de reset, los pines `serial_rx` y `serial_tx`, y los once LED. Su función no es procesar bits ni ejecutar operaciones: instancia los bloques principales, les entrega los mismos parámetros de frecuencia y velocidad, y conecta las interfaces de lectura y escritura entre el núcleo UART y el controlador. Esta capa permite cambiar la implementación de una parte sin alterar los pines físicos del diseño.



Figura 3.2: Interfaz de `alu_uart_top`.

La entrada, al ser externa, puede cambiar sin relación con el reloj. `reset_sync` es el módulo que sincroniza ese cambio. Al presionar el botón de reset, el reset interno se activa de inmediato; al soltarlo, se mantiene activo durante dos flancos y recién entonces se libera. De ese modo todos los módulos abandonan el reset sobre un flanco conocido.



Figura 3.3: Interfaz de `reset_sync`.

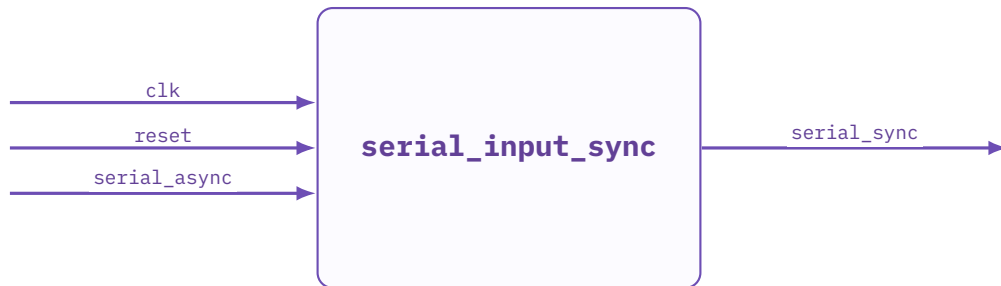
`uart_core` se ocupa de la comunicación serie. A partir de los bits que llegan por `serial_rx`, reconstruye cada byte y lo conserva hasta que el controlador lo retira. En el sentido opuesto, recibe los bytes producidos por el controlador y los transmite por `serial_tx`.

De este modo, el controlador trabaja únicamente con bytes completos. Las señales `rx_empty` y `tx_full` indican cuándo puede leer o escribir, mientras que un pulso separado informa la recepción de una trama inválida. Los detalles propios de UART, como la duración y el orden de los bits o la comprobación del bit de stop, permanecen contenidos dentro del núcleo.

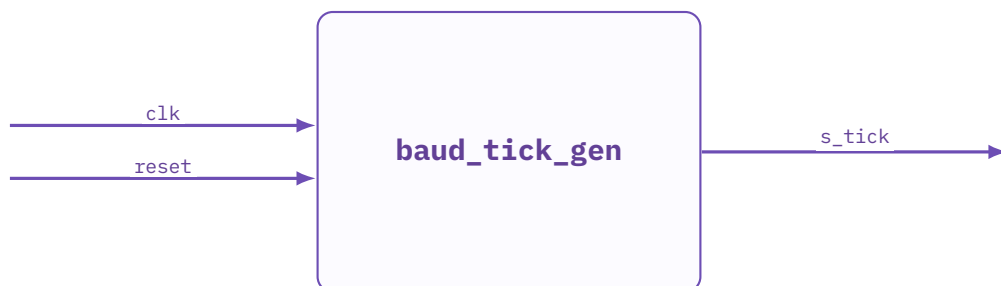
Figura 3.4: Interfaz de `uart_core`.

3.1.2 Camino de recepción

`serial_input_sync` adapta la entrada asíncrona `serial_rx` al reloj interno. Para ello hace pasar la señal por dos flip-flops consecutivos y utiliza solamente la salida del segundo, lo que reduce el riesgo de propagar metastabilidad. La salida sigue siendo un único bit, que luego es procesado por la FSM receptora.

Figura 3.5: Interfaz de `serial_input_sync`.

`baud_tick_gen` divide el reloj del sistema y produce `s_tick`, una habilitación de un ciclo a dieciséis veces la velocidad de baud. No crea un reloj nuevo. RX y TX conservan el reloj de 100 MHz y solo avanzan sus contadores cuando reciben ese pulso. Compartir el generador mantiene una única referencia temporal para ambos sentidos de la comunicación.

Figura 3.6: Interfaz de `baud_tick_gen`.

`uart_rx` es el receptor del enlace UART. Su función es convertir la secuencia recibida por la línea serie en bytes de ocho bits. Cuando completa una trama 8N1 válida, coloca el byte en `data_out` y activa `rx_done_tick` durante un ciclo. Si la trama es inválida, descarta el dato y lo informa mediante `frame_error_tick`. De este modo, los bloques siguientes reciben bytes completos y una indicación explícita de éxito o error.

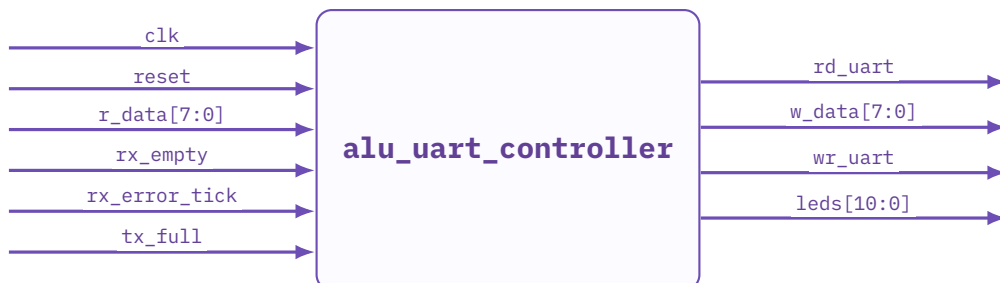
Figura 3.7: Interfaz de `uart_rx`.

`rx_fifo` es la cola de recepción. Su función es conservar los bytes entregados por `uart_rx` hasta que el controlador pueda procesarlos. El byte más antiguo queda disponible en `r_data`; `rx_empty` indica si la cola está vacía y `rd_uart` retira el dato leído. Si se recibe un byte cuando no queda espacio, `uart_core` informa el desbordamiento mediante `rx_error_tick`. Esta cola y la de transmisión son dos instancias del mismo módulo paramétrico `fifo`.

Figura 3.8: Interfaz de la instancia `rx_fifo`.

3.1.3 Interpretación y cálculo

`alu_uart_controller` es el módulo que interpreta el protocolo de la aplicación y coordina la ejecución de la ALU. Recibe bytes completos desde `uart_core`, los organiza como operando A, operando B y opcode, y entrega el resultado al camino de transmisión. También conserva el resultado y las banderas que se muestran en los LED. Si ocurre un error de recepción o vence el timeout antes de completar una solicitud, descarta los datos parciales y espera una transacción nueva.

Figura 3.9: Interfaz de `alu_uart_controller`.

`alu` es el bloque combinacional que realiza las operaciones aritméticas y lógicas. Recibe los operandos A y B junto con los seis bits del opcode, y produce el resultado y las banderas zero, carry y overflow. El controlador registra esas salidas para mantenerlas estables durante la transmisión y en

los LED. De este modo, la ALU se ocupa únicamente del cálculo y no interviene en la comunicación UART.



Figura 3.10: Interfaz de `alu`.

3.1.4 Camino de transmisión y salidas

`tx_fifo` es la cola de transmisión. Almacena los bytes producidos por el controlador hasta que `uart_tx` pueda enviarlos. El controlador escribe mediante `w_data` y `wr_uart`, mientras que `tx_full` indica si existe espacio disponible. Cuando la cola está llena, el controlador conserva el resultado y espera, evitando que se pierda la respuesta.



Figura 3.11: Interfaz de la instancia `tx_fifo`.

`uart_tx` es el transmisor del enlace UART. Su función es convertir un byte completo en una trama 8N1 y enviarla por la salida serie. La señal `tx_start` inicia la transmisión, `tx_busy` indica que el módulo está ocupado y `tx_done_tick` informa que la trama terminó. La salida `tx` se conecta a `serial_tx` en el nivel superior.

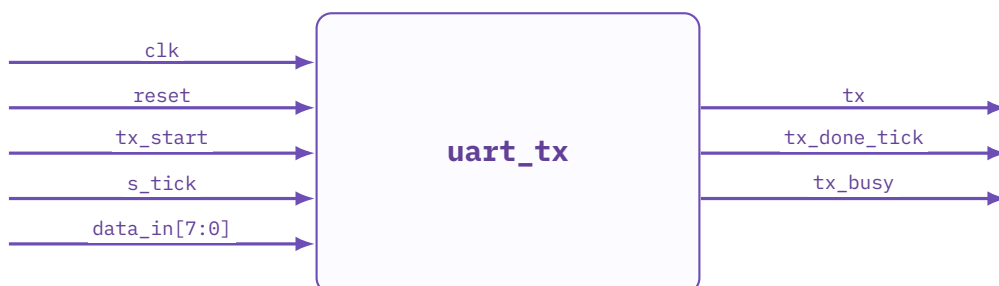


Figura 3.12: Interfaz de `uart_tx`.

Los once LED se controlan de forma independiente al transmisor. Muestran el resultado registrado y las banderas Z, C y V, por lo que conservan su estado aunque la respuesta UART ya haya finalizado.

Esta distribución establece una regla clara entre módulos: el productor solo escribe si la cola no está llena y el consumidor solo lee si no está vacía. Las señales `wr/full` y `rd/empty` convierten una relación temporal implícita en un contrato verificable. Gracias a ello, cada bloque puede probarse de manera aislada antes de comprobar el recorrido completo.

3.2 Flujo de operación

En reposo, las señales RX y TX permanecen en nivel alto, la cola de recepción se encuentra vacía y el controlador espera el operando A. Cuando llega una trama válida, RX entrega el byte recibido a la cola de recepción. El controlador lo retira y pasa a esperar el operando B; el proceso se repite para el opcode. En el estado correspondiente la ALU ve los tres registros estables. Un ciclo después se capturan el resultado y las banderas, y se espera a que haya capacidad suficiente en la cola del transmisor.

La respuesta no se retira de la cola de transmisión cuando comienza a transmitirse, sino al final del stop. En consecuencia, el dato frontal permanece estable durante toda la trama. Si llega un error antes de recibir los tres operandos, el controlador descarta los registros parciales y el próximo byte vuelve a interpretarse como el operando A. Esta regla evita asociar datos pertenecientes a solicitudes distintas.

3.3 Decisiones de diseño

El protocolo con la computadora se mantuvo deliberadamente pequeño. Cada solicitud está formada por tres bytes binarios enviados en un orden fijo: primero el operando A, después el operando B y por último el código de operación. Una vez recibida la terna, el circuito responde con un único byte de resultado. Este formato evita incorporar un analizador de texto, conversiones desde ASCII o caracteres separadores, recursos que no contribuirían al objetivo del trabajo. Como contrapartida, el emisor debe respetar el orden acordado y el controlador debe recordar cuántos bytes válidos recibió antes de ejecutar la ALU.

La UART opera a 19 200 baud, una velocidad muy inferior al reloj de 100 MHz de la placa. Para relacionar ambas velocidades, `baud_tick_gen` cuenta ciclos del reloj principal y activa `s_tick` cada 326 ciclos. De esta forma se obtienen aproximadamente dieciséis instantes de referencia durante cada bit de la trama UART.

El receptor y el transmisor continúan sincronizados con el reloj de 100 MHz, pero utilizan `s_tick` para determinar cuándo avanzar. Por lo tanto, `s_tick` es una señal de habilitación y no un segundo reloj. El receptor aprovecha esas referencias para verificar el inicio de la trama y leer cada bit cerca de su centro.

La recepción, la transmisión y la interpretación de una solicitud avanzan por etapas y deben recordar cuál es la siguiente acción. Por eso cada una se controla mediante una máquina de estados: RX, TX y el controlador. Las tres utilizan codificación one-hot. Cada estado posee su propio flip-flop: RX emplea cinco, TX cuatro y el controlador otros cinco. Frente a una codificación binaria se consumen más registros, pero en esta FPGA el costo es reducido y se obtiene una relación directa entre cada estado y su bit de representación. Esto simplifica el seguimiento de las transiciones durante la simulación y permite reconocer con facilidad el estado activo en los reportes de síntesis. El atributo `fsm_encoding = "one_hot"` deja explícita esa elección, y la rama por defecto de cada FSM define además un retorno seguro si el registro de estado adopta una combinación inválida.

Los FIFO resuelven la diferencia de ritmo entre la comunicación UART y el procesamiento interno. Se eligió una profundidad de cuatro bytes porque alcanza para absorber una ráfaga breve sin convertir las colas en memoria para una sesión completa. En recepción, cada trama válida ocupa una posición hasta que el controlador la retira; si llega un nuevo byte con las cuatro posiciones ocupadas, el núcleo no sobrescribe información previa y notifica el desbordamiento (*overflow*). En transmisión, la situación se resuelve mediante espera: si `tx_full` está activo, el controlador conserva el resultado y sus banderas hasta que exista una posición libre. El registro de respuesta también mantiene estables los LED y evita que una solicitud parcial modifique el último resultado válido.

El orden fijo de tres bytes plantea un último problema: si la computadora envía A o A–B y luego interrumpe la transferencia, el controlador no puede conservar esa solicitud incompleta de manera indefinida. Por ese motivo se incorporó un tiempo máximo de espera (*timeout*) de 10 ms. A 19 200 baud, una trama 8N1 dura aproximadamente 0,521 ms, de modo que el margen admite pausas considerablemente mayores que el tiempo de transmisión normal. Si el plazo vence o RX informa un error antes de completar la terna, se descartan los registros parciales y el siguiente byte vuelve a interpretarse como el operando A.

En conjunto, estas decisiones priorizan un comportamiento determinista y verificable. Cada bloque posee una condición explícita para aceptar datos, esperar o recuperarse, sin depender de retardos implícitos entre módulos.

3.4 Organización del proyecto

El árbol de la figura 3.13 refleja la jerarquía sintetizable, no solo la distribución de archivos. `alu_uart_top` instancia el acondicionador de reset, el núcleo UART y el controlador. Dentro del núcleo quedan sincronizador, generador, receptor, transmisor y ambos FIFO; dentro del controlador se conserva la ALU del TP1.

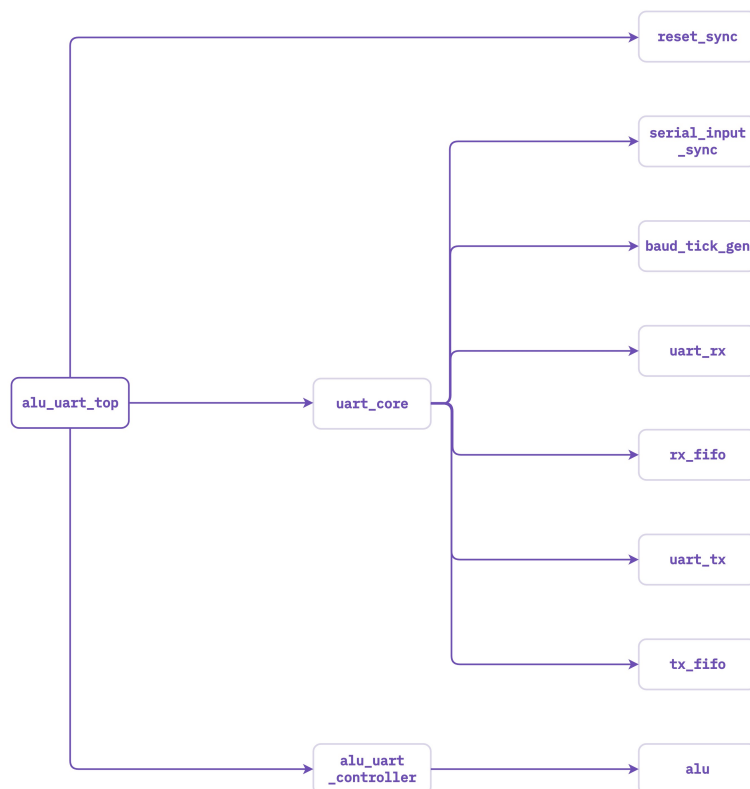


Figura 3.13: Jerarquía de módulos elaborada a partir de las instancias RTL.

Sección 4

Núcleo UART y máquinas de estado

4.1 Contrato del enlace y formato UART 8N1

En el TP1, cada operando llegaba a la ALU mediante ocho señales paralelas: una por cada bit. En este trabajo, la computadora envía esos mismos ocho bits de forma secuencial mediante UART. Los bits de cada byte ingresan uno después de otro por RX y la respuesta se transmite del mismo modo por TX. Como UART no incorpora una señal de reloj, la computadora y la FPGA deben utilizar la misma velocidad y el mismo formato de trama. En este diseño se emplean 19 200 baud y el formato 8N1: ocho bits de datos, sin paridad y con un bit de stop.

Cada byte ocupa diez períodos de bit. Mientras no hay una transmisión, RX y TX permanecen en nivel alto. Un bit de start en bajo marca el comienzo de la trama; a continuación se envían los ocho bits de datos, desde el menos significativo hasta el más significativo, y un bit de stop en alto indica el final.

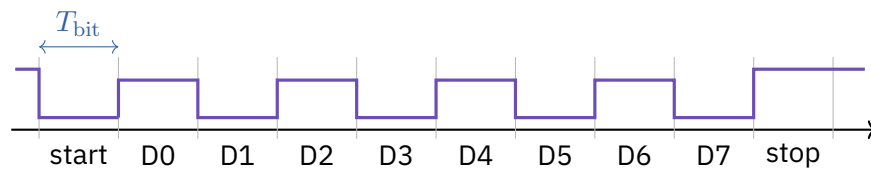


Figura 4.1: Estructura temporal de una trama UART 8N1.

Las líneas verticales de la figura 4.1 delimitan diez intervalos iguales. El primer nivel bajo no es parte del dato: permite al receptor detectar el inicio. D0 llega antes que D7, por lo que el orden temporal no coincide con la forma habitual de escribir un byte. El stop alto confirma el final y devuelve la línea a su nivel de reposo.

A 19 200 baud, un bit dura aproximadamente $52,083 \mu s$ y cada byte ocupa $520,833 \mu s$. El envío de los tres bytes de entrada demanda al menos $1,563 ms$; la respuesta agrega otro intervalo de byte. Estos valores no incluyen demoras del sistema operativo o del adaptador USB.

4.2 Generación del tick de muestreo

El receptor utiliza sobremuestreo 16x: por cada bit recibido se generan dieciséis pulsos de muestreo. De este modo, cada período de bit queda dividido en dieciséis intervalos que sirven como referencia temporal. Después de detectar la transición que puede indicar un bit de start, el receptor espera ocho pulsos para comprobar su valor cerca del centro. Si el start es válido, toma una muestra de cada bit de datos cada dieciséis pulsos y finalmente hace lo mismo con el bit de stop.

Para obtener esos dieciséis pulsos por bit, el generador produce `s_tick` a una frecuencia 16 veces mayor que la tasa UART:

$$f_{tick} = 16 \cdot 19200 = 307\,200 \text{ Hz.} \quad (4.1)$$

Como el reloj de la placa es de 100 MHz, se utiliza un divisor entero redondeado al valor más próximo:

$$D = \text{round} \left(\frac{100\,000\,000}{307\,200} \right) = 326. \quad (4.2)$$

El resultado no es exacto. El tick real es 306 748,466 Hz y la tasa efectiva es 19 171,779 baud, con un error relativo de aproximadamente $-0,147\%$. La diferencia acumulada a lo largo de una trama es muy inferior a medio bit y queda dentro del margen que ofrece el muestreo central.

El RTL conserva el reloj de 100 MHz y genera un pulso de habilitación cada 326 ciclos. La suma de media frecuencia antes de la división implementa el redondeo entero.

```
localparam integer TICK_FREQ_HZ = BAUD_RATE * OVERSAMPLE;
localparam integer DIVISOR =
    (CLK_FREQ_HZ + (TICK_FREQ_HZ / 2)) / TICK_FREQ_HZ;

always @(posedge clk) begin
    if (reset) begin
        counter <= '0;
        s_tick <= 1'b0;
    end else if (counter == DIVISOR - 1) begin
        counter <= '0;
        s_tick <= 1'b1;
    end else begin
        counter <= counter + 1'b1;
        s_tick <= 1'b0;
    end
end
```

El fragmento muestra qué hardware se obtiene: un contador, un comparador contra 325 y un registro para el pulso. Como `s_tick` permanece bajo en todos los demás ciclos, RX y TX avanzan sus contadores de sub-bit únicamente cuando corresponde, pero todos sus registros siguen capturados por `clk`.

4.3 Protocolo de aplicación

Cada operación tiene longitud fija. La computadora transmite A, luego B y por último el opcode. Una vez recibidos los tres bytes válidos, la FPGA responde con un byte de resultado.

Sentido	Byte 0	Byte 1	Byte 2	Respuesta
PC a FPGA	A, 8 bits	B, 8 bits	Opcode en bits 5–0	—
FPGA a PC	—	—	—	Resultado, 8 bits

Tabla 4.1: Orden de los bytes del protocolo binario.

Los bits 7 y 6 del tercer byte se ignoran. Esta elección permite enviar el opcode dentro de una palabra UART completa sin agregar codificación ASCII. El protocolo es deliberadamente mínimo:

no se devuelve un byte separado de estado porque el requisito funcional solicita el resultado y las banderas continúan visibles en LED8, LED9 y LED10.

Operación	Opcode	Hexadecimal	Resultado
SRA	000011	03	A desplazado aritméticamente B lugares
SRL	000010	02	A desplazado lógicamente B lugares
ADD	100000	20	A + B
SUB	100010	22	A - B
AND	100100	24	A AND B
OR	100101	25	A OR B
XOR	100110	26	A XOR B
NOR	100111	27	NOT (A OR B)

Tabla 4.2: Operaciones conservadas del TP1.

Por ejemplo, para calcular $7F + 01$, el host transmite $7F\ 01\ 20$. La respuesta esperada es 80 ; al mismo tiempo, LED10 se enciende para indicar overflow y los LED de carry y cero permanecen apagados.

4.4 Recuperación y casos inválidos

El controlador debe volver a su estado inicial si la computadora deja de enviar datos antes de completar una solicitud. Una vez recibido A, o A y B, espera como máximo 10 ms el byte siguiente. Si vence ese plazo, descarta la operación parcial y el próximo byte se interpreta nuevamente como A. El timeout supera ampliamente los $0,521\text{ ms}$ de una trama normal, aun cuando los bytes se envían de manera individual.

Situación	Acción	Respuesta
Start vuelve a alto antes del centro	Abortar la trama y esperar reposo alto.	Ninguna.
Stop bajo	Emitir error de trama y esperar reposo alto.	Ninguna.
FIFO RX lleno	Conservar los cuatro bytes previos y notificar overrun.	Se aborta la transacción parcial.
Transacción incompleta	Vencer el contador a los 10 ms.	Ninguna.
Opcode no reconocido	Ejecutar el caso por defecto de la ALU.	Un byte 00, con Z activo.
FIFO TX lleno	Conservar resultado e indicadores.	Esperar espacio antes de encolar.

Tabla 4.3: Política de tratamiento de condiciones excepcionales.

4.5 Estructura y codificación de las máquinas de estado

Como se anticipó al describir la arquitectura, la recepción, la transmisión y el control de cada operación avanzan por etapas y necesitan conservar su progreso entre ciclos de reloj. Por ese motivo se utilizaron tres máquinas de estado: RX, TX y el controlador.

Las tres FSM comparten la misma estructura: un registro de estado, lógica combinacional para determinar el próximo estado y las salidas, constantes locales para representar cada etapa y una rama `default` que permite regresar a IDLE ante una codificación no contemplada.

Se eligió la codificación one-hot para las tres FSM. En este tipo de codificación cada estado ocupa un flip-flop y solo un bit debe permanecer activo. Esta representación consume más registros que una codificación binaria, pero simplifica la lógica de decodificación en FPGA y hace visibles los estados durante la depuración. RX utiliza cinco bits, TX cuatro y el controlador cinco. El atributo `fsm_encoding = one_hot` conserva esta intención durante la extracción y optimización de las máquinas en Vivado.

En una codificación binaria cinco estados requerirían tres bits y cada transición debería decodificar una combinación. En la codificación one-hot se reservan cinco bits, por ejemplo 00001 para IDLE y 00100 para DATA. El costo adicional es de dos flip-flops; la ventaja es que comprobar “estoy en DATA” se reduce conceptualmente a observar un solo bit. Esto no elimina la necesidad de describir qué ocurre ante 00000 o ante dos bits activos: la rama `default` devuelve la FSM al estado IDLE.

```

(* fsm_encoding = "one_hot" *) reg [4:0] state = STATE_IDLE;
reg [4:0] next_state;

always @(posedge clk) begin
    if (reset)
        state <= STATE_IDLE;
    else
        state <= next_state;
end

always_comb begin
    next_state = state;
    case (state)
        // cada estado modifica solo las senales necesarias
        default: next_state = STATE_IDLE;
    endcase
end

```

El registro secuencial `state` representa el estado actual. El bloque combinacional calcula el próximo estado `next_state` y las salidas a partir de entradas y registros.

4.6 Receptor UART

La FSM del receptor separa reposo, validación del bit de start, recepción de los datos, validación del bit de stop y recuperación. Sus transiciones principales se resumen en la figura 4.2.

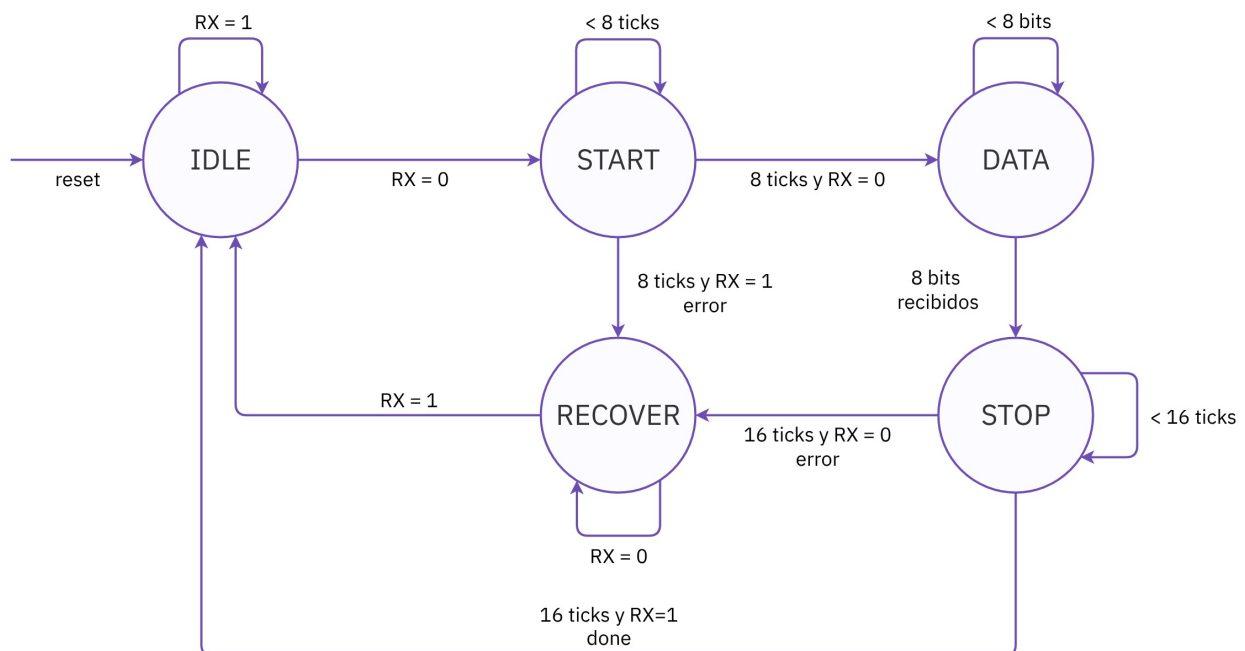


Figura 4.2: Estados y condiciones principales del receptor.

4.6.1 Validación del bit de start

En IDLE, una entrada baja inicia la observación. La FSM no la acepta de inmediato si no que espera ocho ticks, es decir, medio período de bit para sobremuestreo 16x. Si RX continúa bajo, el punto observado corresponde al centro del start y se pasa a DATA. Si volvió a alto, se trata de un pulso espurio por lo que se genera un único `frame_error_tick` y se ingresa en RECOVER.

4.6.2 Muestreo y orden de bits

Después del centro del start, cada dato se toma al completar dieciséis ticks. Como UART envía D0 primero, la nueva muestra entra por el bit más significativo de un registro que se desplaza a la derecha:

```
next_data = {rx, data_reg[DATA_BITS-1:1]};
```

Tras ocho iteraciones, las muestras quedan en su posición natural. Se espera entonces un período completo de stop. Solo si RX es alto se genera `rx_done_tick`; un stop bajo no publica el byte.

4.6.3 Estado de recuperación

Después de un falso start o de un stop inválido, la línea puede permanecer baja durante varios bits. Si se volviera inmediatamente a IDLE se generaría un error repetido por cada nueva observación de ese mismo nivel. RECOVER mantiene los contadores en cero y espera el retorno a reposo en alto antes de habilitar otra trama. Así, cada evento defectuoso produce un solo pulso de error.

4.7 Transmisor UART

El transmisor recibe un byte completo, pero debe enviarlo por `tx` un bit a la vez y respetar la duración de cada parte de la trama 8N1. Una máquina de estados conserva el avance de esa secuencia. Como todos los niveles de salida son generados por el propio circuito, se requieren cuatro estados: IDLE, START, DATA y STOP.

En IDLE, la línea permanece en alto. Un pulso en `tx_start` hace que el módulo guarde `data_in` y comience el bit de start en nivel bajo. Luego transmite los ocho bits de datos desde D0 hasta D7 y finaliza con el bit de stop en alto. Cada bit se mantiene durante dieciséis pulsos de `s_tick`.

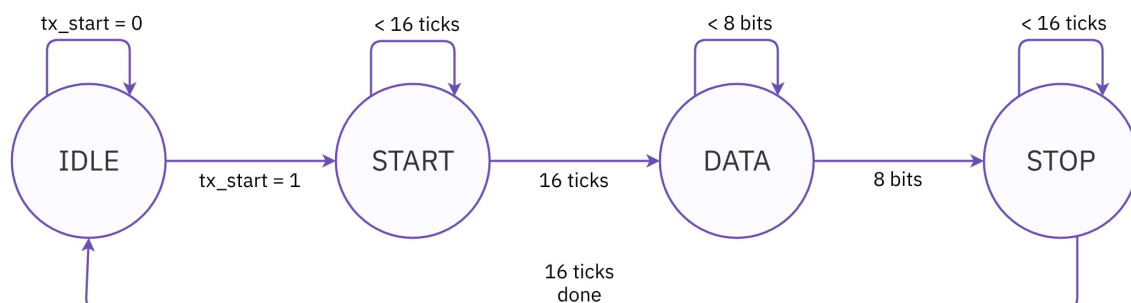


Figura 4.3: Secuencia de estados del transmisor.

Mientras la trama está en curso, `tx_busy` permanece activo. Al completar el stop, `tx_done_tick`

se activa durante un ciclo y permite que el núcleo retire el byte del FIFO. Si todavía quedan datos pendientes, una nueva activación de `tx_start` inicia la trama siguiente.

El byte se copia a un registro interno al comenzar la transmisión. Así conserva el mismo valor durante toda la trama, aunque `data_in` cambie antes de que finalice. Al terminar cada período de bit, el registro se desplaza para presentar el dato siguiente:

```
if (tx_start) begin
    next_state = STATE_START;
    next_data  = data_in;
    next_tx    = 1'b0;
end
// ... al terminar cada dato
next_data = {1'b0, data_reg[DATA_BITS-1:1]};
next_tx   = data_reg[1];
```

4.8 Buffers FIFO

La UART y el controlador no siempre están listos en el mismo ciclo. El FIFO se ubica entre ambos y conserva temporalmente los bytes, de modo que el productor puede escribir sin esperar que el consumidor los retire de inmediato. Una escritura se acepta mientras exista espacio y una lectura, mientras haya datos disponibles.

Cada FIFO puede almacenar cuatro bytes. El registro `read_pointer` señala el byte que se leerá a continuación, mientras que `write_pointer` indica la posición de la próxima escritura. Ambos avanzan de forma circular: después de la última posición regresan a la primera.

El registro `count` indica cuántos bytes contiene la cola. Este valor permite distinguir entre vacío y lleno incluso cuando los dos punteros coinciden. Si una lectura y una escritura se aceptan en el mismo ciclo, sale un byte y entra otro, por lo que la cantidad de datos almacenados no cambia.

```
assign empty = (count == 0);
assign full  = (count == DEPTH);
assign read_accept = rd && !empty;
assign write_accept = wr && !full;

case ({write_accept, read_accept})
    2'b10: count <= count + 1'b1;
    2'b01: count <= count - 1'b1;
    default: count <= count;
endcase
```

Si la cola está llena al comenzar el ciclo, no puede aceptar una nueva escritura, aunque `rd` y `wr` se activen al mismo tiempo. En ese flanco se realiza solamente la lectura, que libera una posición. La escritura puede repetirse en el ciclo siguiente, cuando ya existe espacio disponible. De esta forma se evita sobrescribir el dato más antiguo. Este caso también se comprueba en `fifo_tb`.

4.9 Controlador de transacciones

La tercera FSM traduce bytes completos al contrato de la ALU. En el estado WAIT_A consume el primer elemento de la cola de recepción y carga el operando A. En los estados WAIT_B y WAIT_OPCODE se hace lo mismo para el operando B y el opcode. El estado de EXECUTE reserva un ciclo para registrar el resultado y mostrar los indicadores. El estado de WAIT_TX conserva esos registros si la cola de transmisión está llena y escribe cuando existe espacio en la misma.

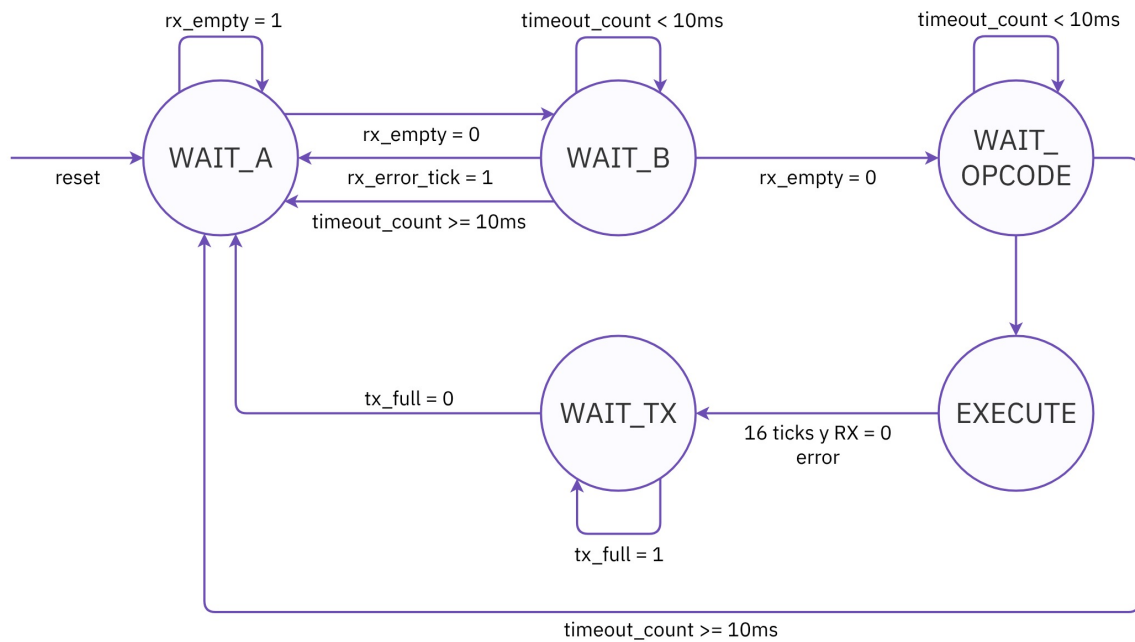


Figura 4.4: FSM que delimita una operación de tres bytes.

El timeout solo es necesario después de recibir el primer operando y antes de completar la solicitud. Por eso el contador avanza mientras el controlador espera B o el opcode, y vuelve a cero cada vez que llega un byte válido. Mientras se espera A permanece inactivo. Si vence el plazo o RX informa un error durante una solicitud parcial, los datos recibidos se descartan y el controlador vuelve a WAIT_A.

Cuando llega el opcode, su registro se actualiza al finalizar el flanco de reloj. El resultado de la ALU no puede capturarse en ese mismo instante porque todavía depende del valor anterior. El estado EXECUTE proporciona un ciclo adicional para que A, B y opcode permanezcan estables y la ALU produzca el resultado correcto. Después, WAIT_TX conserva ese resultado y sus banderas hasta que haya espacio en la cola de transmisión.

```
STATE_EXECUTE: begin
    capture_result = 1'b1;
    next_state = STATE_WAIT_TX;
end

STATE_WAIT_TX: begin
    if (!tx_full) begin
        wr_uart = 1'b1;
        next_state = STATE_WAIT_A;
    end
end
```

4.10 Recuperación de estados ilegales

En una codificación one-hot, solo son válidas las combinaciones que tienen un único bit activo. Una perturbación podría modificar el registro y producir una combinación que no represente ningún estado definido. Para evitar que la máquina quede detenida en esa situación, cada sentencia `case` incluye una rama `default`.

Ante un estado no reconocido, las FSM de RX y TX regresan a IDLE y restablecen sus registros auxiliares. El controlador vuelve a WAIT_A para comenzar una solicitud nueva. De este modo, todas las combinaciones posibles conducen a un comportamiento definido y la operación puede continuar desde un estado conocido en el ciclo siguiente.

Sección 5

Integración con la ALU y la Basys 3

El sistema se utiliza desde una computadora conectada al puerto USB–UART de la Basys 3. Desde allí se envían los operandos y el opcode de cada operación. La placa procesa esos valores y devuelve el resultado por el mismo enlace.

Los datos no pasan directamente desde la computadora hasta la ALU. Primero se transmiten por UART como una secuencia de tres bytes. El controlador recupera esos valores y los presenta en paralelo a la ALU. Luego, el resultado vuelve a la computadora y también queda visible en los LED.

La integración conecta los bloques que participan en ese recorrido. En este nivel se establece cómo se transfieren los datos, cuándo se acepta cada byte y qué información se presenta fuera de la FPGA. Cada módulo mantiene una función propia y se comunica con los demás mediante señales bien definidas.

5.1 Recorrido de una transacción

Cada solicitud comienza con el envío del operando A, el operando B y el opcode. `uart_core` recibe las tres tramas y almacena los bytes reconstruidos. El controlador los retira en el mismo orden y guarda cada uno en el registro correspondiente.

Cuando la solicitud está completa, la ALU recibe ambos operandos y los seis bits inferiores del opcode. El controlador registra el resultado y las banderas en el ciclo siguiente. El resultado se entrega entonces a la cola de transmisión y los LED se actualizan con la información de la operación.

La tabla 5.1 resume este recorrido. Cada transferencia entre bloques se realiza en un flanco del reloj.

Paso	Transferencia	Efecto
0	Reset hacia todos los bloques	Las colas quedan vacías y el resultado inicial toma el valor cero.
1	Computadora hacia <code>uart_core</code>	Las tramas de A, B y opcode se reconstruyen como bytes.
2	FIFO RX hacia el controlador	Los tres campos se guardan en el orden definido por el protocolo.
3	Controlador hacia la ALU	A, B y opcode quedan disponibles en paralelo.
4	ALU hacia el controlador	El resultado y las banderas se registran y actualizan los LED.
5	Controlador hacia FIFO TX	El byte de resultado queda pendiente de transmisión.
6	<code>uart_tx</code> hacia la computadora	El resultado se serializa y se envía por la línea UART.

Tabla 5.1: Recorrido de una operación a través del sistema integrado.

El controlador vuelve a esperar una solicitud después de registrar la operación anterior. De esta manera, el byte transmitido y el patrón de los LED corresponden a los mismos operandos y al mismo opcode.

5.2 Resultado UART e indicadores

La respuesta UART contiene solamente los ocho bits del resultado. Las banderas Z, C y V no se transmiten. En cambio, se muestran junto con el resultado en los once LED de la placa.

El controlador registra todos estos valores al completar una operación. La salida hacia los LED se forma a partir de esos registros según la siguiente distribución:

$$\text{leds}[10:0] = \{V, C, Z, \text{resultado}[7:0]\}. \quad (5.1)$$

Los valores registrados se mantienen hasta que termina una nueva operación. Esto permite observar el resultado aunque la respuesta UART ya haya sido transmitida. Después del reset, los ocho bits de resultado quedan en cero. La bandera Z queda activa y las banderas C y V permanecen apagadas.

5.3 Coordinación y rendimiento

Los bloques no siempre producen y consumen los bytes en el mismo ciclo. La cola de recepción conserva los datos que todavía no fueron leídos por el controlador. La cola de transmisión cumple la misma función con los resultados que todavía no fueron enviados.

El controlador solo retira un byte cuando la cola de recepción contiene datos. También comprueba que exista espacio antes de guardar una respuesta. Si la cola de transmisión está llena, conserva el

resultado y espera hasta que pueda escribirlo. Así, una diferencia momentánea entre los bloques no provoca la pérdida de información.

La velocidad máxima queda determinada por el enlace UART. Cada trama 8N1 ocupa diez bits. A 19 200 baud pueden transmitirse 1920 bytes por segundo. Como cada solicitud contiene tres bytes, la entrada admite como máximo 640 operaciones por segundo.

Cada operación genera un solo byte de respuesta. Por lo tanto, esa cantidad de solicitudes requiere 6400 bits por segundo en el sentido de transmisión. Este valor se encuentra por debajo de la capacidad disponible. Además, las dos líneas UART permiten recibir una solicitud mientras se transmite el resultado anterior.

La aplicación de la computadora espera la respuesta de una operación antes de enviar la siguiente. Con esta forma de intercambio, cada resultado puede asociarse directamente con la solicitud que lo produjo.

5.4 Interfaz gráfica de la computadora

Para operar y validar el sistema se desarrolló una aplicación gráfica en Python. La herramienta permite seleccionar el puerto serie, ingresar los dos operandos en distintos formatos y elegir la operación que debe ejecutar la ALU.

Al iniciar una operación, la aplicación envía la solicitud a la placa mediante UART y muestra el resultado recibido en hexadecimal, binario y decimal. Además, lo compara con un modelo de referencia calculado localmente. Las banderas Z, C y V que presenta la interfaz también provienen de este modelo y sirven como referencia para comprobar los LED de la placa, ya que no se transmiten en la respuesta UART.

La aplicación incluye asimismo una vista de pruebas que permite ejecutar casos dirigidos y pseudoaleatorios reproducibles. De este modo, funciona tanto como medio de operación de la ALU como herramienta de apoyo para la validación funcional del sistema. La figura 5.1 muestra su vista principal durante la ejecución de una suma.

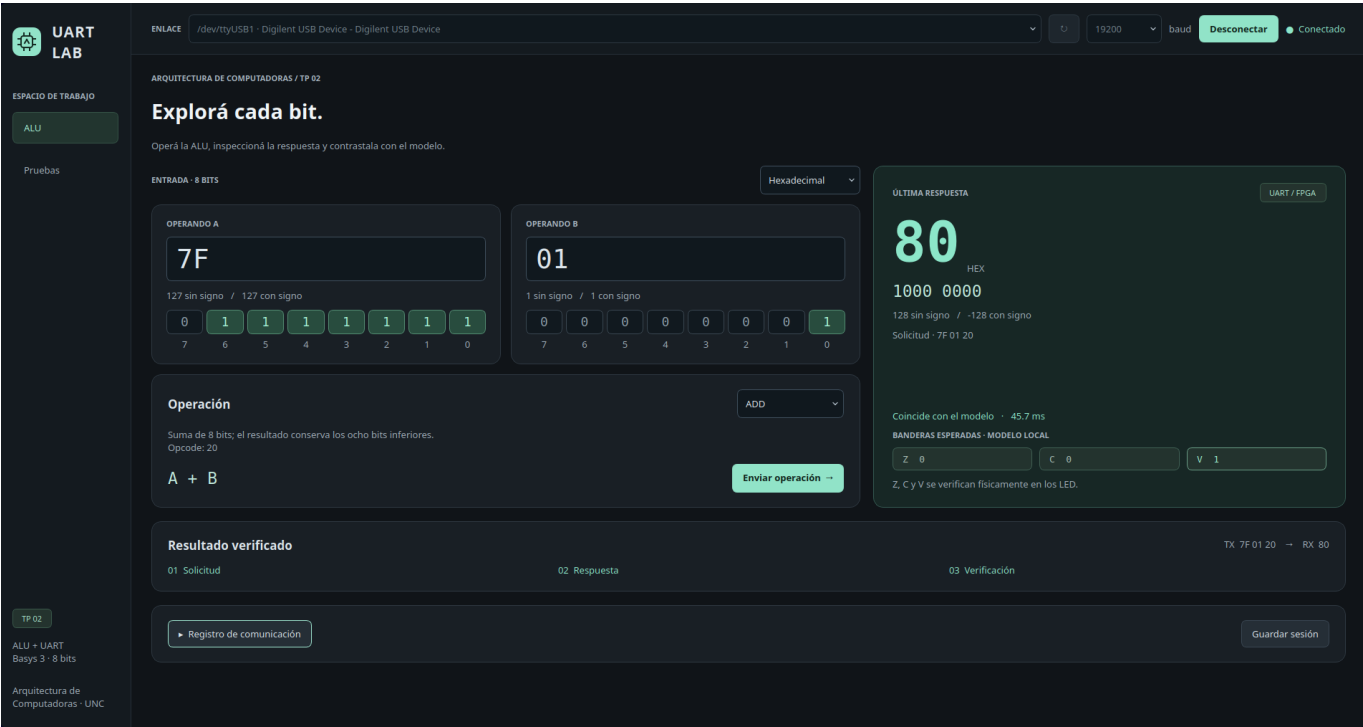


Figura 5.1: Interfaz gráfica utilizada para operar la ALU y comparar la respuesta recibida con el modelo de referencia.

5.5 Integración con la Basys 3

El módulo superior `alu_uart_top.sv` conecta el diseño con los recursos de la placa. Sus puertos corresponden al reloj de entrada, el pulsador de reset, las dos líneas UART y los once LED. El archivo `basys3.xdc` de restricciones asigna estas señales a los pines de la Basys 3.

Puerto	Pin	Recurso físico
<code>clk</code>	W5	Oscilador de 100 MHz.
<code>btn_reset</code>	U18	Pulsador central BTNC.
<code>serial_rx</code>	B18	USB–UART desde la computadora.
<code>serial_tx</code>	A18	USB–UART hacia la computadora.
<code>leds[7:0]</code>	varios	Resultado de ocho bits.
<code>leds[10:8]</code>	W3, V3, V13	Banderas V, C y Z.

Tabla 5.2: Correspondencia entre los puertos del diseño y la Basys 3.

Las señales `serial_rx` y `btn_reset` se originan fuera de la FPGA y pueden cambiar en cualquier momento respecto del reloj interno. Por este motivo, ambas reciben un tratamiento especial antes de utilizarse en el resto del circuito.

La entrada `serial_rx` atraviesa el sincronizador ubicado antes del receptor UART. En el caso

del reset, la activación es inmediata para llevar los bloques a un estado conocido. Su liberación se sincroniza con el reloj para que todos retomen el funcionamiento a partir de un flanco válido.

Estas características también se tuvieron en cuenta al definir las restricciones temporales. El archivo `basys3.xdc` establece un período de 10 ns para el reloj y aplica las excepciones únicamente al recorrido de las entradas asíncronas hasta sus sincronizadores. A partir de ese punto, las señales se analizan del mismo modo que el resto de la lógica interna. El archivo también establece el estándar eléctrico LVCMOS33 para los puertos de la placa.

Con estas conexiones, el resultado de una operación puede comprobarse de dos formas. La computadora recibe el byte transmitido por UART y la placa muestra ese mismo valor junto con las banderas.

Sección 6

Verificación funcional

6.1 Criterio de validación

La verificación funcional se organizó de acuerdo con los niveles del diseño. Primero se probaron los bloques elementales de forma aislada. Luego se verificaron el núcleo UART y el controlador. Por último, se comprobó el recorrido completo entre los pines de recepción y transmisión. Este orden permitió revisar cada función antes de evaluar la integración del sistema.

Todos los bancos de pruebas son autocheckeables. Cada uno genera sus estímulos y determina el valor esperado mediante un modelo independiente o una secuencia dirigida. Después compara ese valor con la salida del diseño mediante una desigualdad estricta. Si encuentra una diferencia, incrementa el contador de errores y finaliza la simulación con `$fatal`.

Cada banco incorpora también un watchdog que limita la duración de la simulación. Si el diseño queda esperando indefinidamente, el banco informa el error en lugar de considerar que la prueba terminó correctamente.

El resultado PASS confirma que el banco terminó sin diferencias, pero no indica por sí solo qué comportamiento fue comprobado. Por eso, cada resultado se relaciona con los estímulos aplicados y con las señales observadas.

La tabla 6.1 relaciona cada requisito con el banco que lo verifica y con la evidencia obtenida. Las formas de onda incluidas más adelante muestran las señales necesarias para seguir los estímulos y las respuestas. Los casos restantes se comprueban mediante las comparaciones automáticas de cada banco. En cada nivel se presentan el criterio utilizado, el resultado observado y la interpretación de esos resultados.

Requisito	Banco	Evidencia
Conservar la ALU	<code>alu_tb</code>	814 comparaciones y ocho operaciones.
Tick periódico	<code>baud_tick_gen_tb</code>	Intervalo exacto y reset.
Buffer ordenado	<code>fifo_tb</code>	Lleno, vacío, bloqueo y wrap-around.
Recibir 8N1	<code>uart_rx_tb</code>	Bytes, falso start y stop inválido.
Transmitir 8N1	<code>uart_tx_tb</code>	Orden LSB, duración y finalización.
Integrar UART/FIFO	<code>uart_core_tb</code>	Ráfagas, orden y overrun.
Interpretar la terna	<code>alu_uart_controller_tb</code>	EXECUTE, timeout, error RX y backpressure.
Recorrido pin a pin	<code>alu_uart_top_tb</code>	812 transacciones, reset, recuperación y LED.

Tabla 6.1: Trazabilidad entre requisitos y bancos de pruebas.

6.2 Bloques elementales: ALU, temporización, FIFO y UART

El primer nivel de verificación estudia por separado los bloques que realizan las funciones básicas del sistema. De esta manera se puede comprobar el cálculo de la ALU, la generación de los ticks, el

comportamiento del FIFO y la formación de las tramas UART antes de conectarlos entre sí.

6.2.1 Criterio y estímulos

El archivo `tb/alu_tb.sv` verifica que la ALU conserve el comportamiento definido para sus ocho operaciones. El banco aplica catorce casos dirigidos y cien pares pseudoaleatorios por cada operación. La secuencia aleatoria utiliza la semilla fija `32'h1a2b3c4d` para que pueda repetirse.

El modelo de referencia calcula el resultado y las banderas carry y overflow con expresiones diferentes de las utilizadas por el DUT. La bandera Z se obtiene a partir del resultado esperado. En total, el banco realiza 814 comparaciones.

La prueba del generador se encuentra en `tb/aud_tick_gen_tb.sv`. Para reducir el tiempo de simulación se utilizan una frecuencia de 1000 Hz, una velocidad de 25 baud y un sobremuestreo 4x. Estos valores mantienen el divisor en diez y permiten comprobar tres intervalos completos en pocos ciclos. El banco también activa el reset y verifica que no se produzca ningún pulso espurio.

El archivo `tb/fifo_tb.sv` comprueba el orden de los datos y el comportamiento de la cola en sus límites. Primero escribe los valores `11`, `22`, `33` y `44` hasta completar las cuatro posiciones. A continuación intenta escribir `99` y comprueba que el dato sea rechazado.

Después de retirar parte del contenido, el banco escribe `55` y `66`. Con esta secuencia obliga a los punteros a volver al comienzo de la memoria. También aplica una lectura y una escritura durante el mismo ciclo con el FIFO lleno. Cada dato extraído se compara con el orden en que fue insertado.

La recepción se verifica mediante `tb/uart_rx_tb.sv`. El banco envía las tramas correspondientes a `00`, `FF`, `A5` y `3C`. Luego mantiene la entrada en bajo durante cuatro ticks para producir un falso start. También envía una trama cuyo bit de stop permanece en bajo. Después de estos errores transmite `69` para comprobar que el receptor pueda volver a aceptar una trama válida.

El archivo `tb/uart_tx_tb.sv` verifica el recorrido opuesto. El transmisor recibe los bytes `A5`, `00` y `FF`. El banco observa la línea durante las dieciséis muestras de cada bit, comprueba que D0 se envíe primero y exige un único pulso de finalización por cada trama.

6.2.2 Resultado observado

La figura 6.1 muestra la recepción del byte `A5`. La línea serie presenta primero el bit de start y luego los ocho bits de datos. Durante ese intervalo, la FSM permanece en DATA y `bit_count` avanza con cada muestra aceptada. El registro interno reconstruye el byte mediante desplazamientos. El pulso `done` aparece recién después de comprobar el bit de stop y el valor publicado es `A5`.

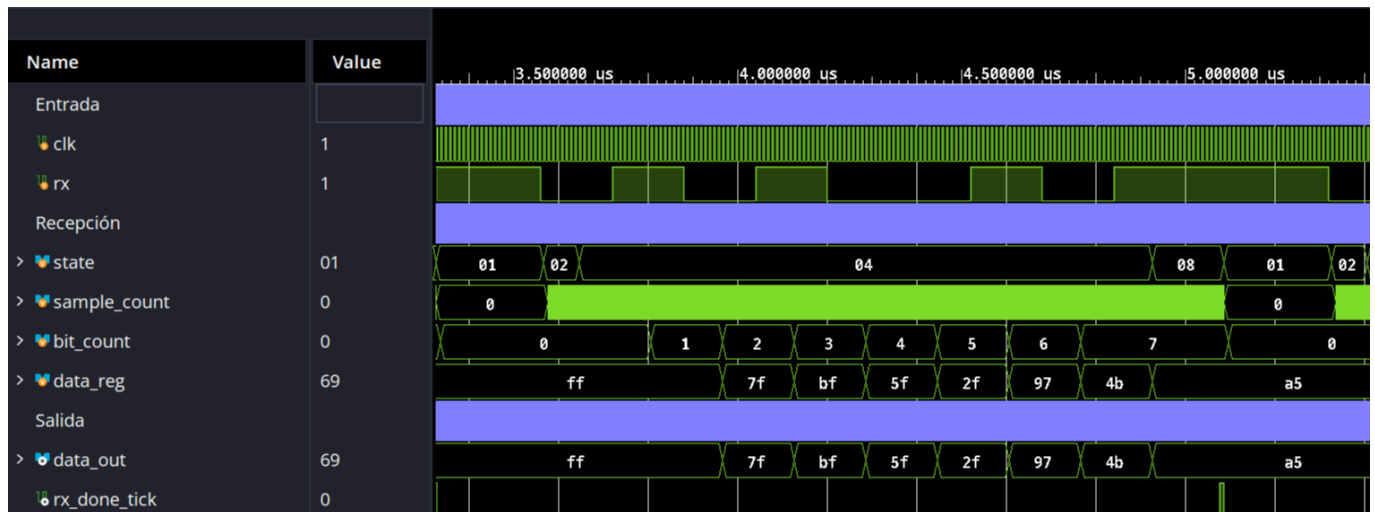


Figura 6.1: Recepción válida de A5 en `uart_rx_tb`.

La figura 6.2 reúne las dos tramas defectuosas. El pulso breve de la entrada produce el primer `frame_error`. La trama siguiente alcanza el estado DATA, pero el nivel bajo durante el stop produce el segundo error. En ninguno de los casos se activa `done`.

Después de detectar cada defecto, el estado RECOVER espera que la entrada RX regrese al nivel alto antes de volver a IDLE. Por esta razón, cada trama defectuosa genera un solo pulso de error.

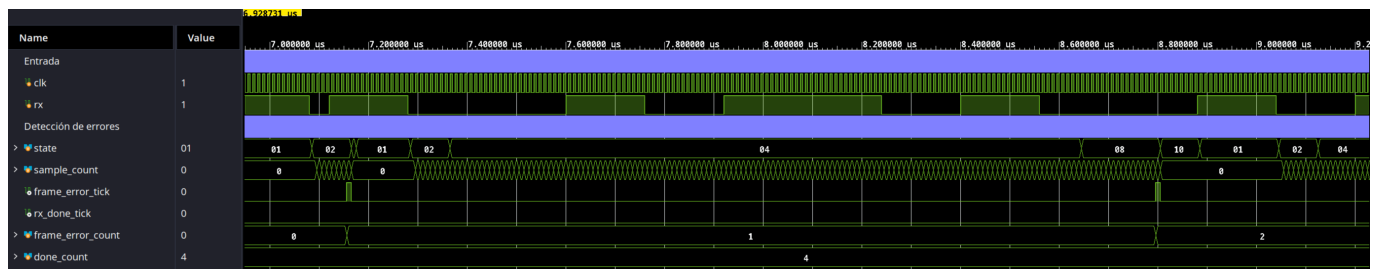


Figura 6.2: Detección de falso start y stop inválido en `uart_rx_tb`.

La figura 6.3 presenta la transmisión de $A5 = 1010\ 0101$. Después del start en nivel bajo, la línea muestra la secuencia 1, 0, 1, 0, 0, 1, 0, 1. Este orden corresponde al envío desde D0 hasta D7. La señal `busy` permanece activa durante toda la trama y `done` se mantiene en alto durante un solo ciclo después del stop.

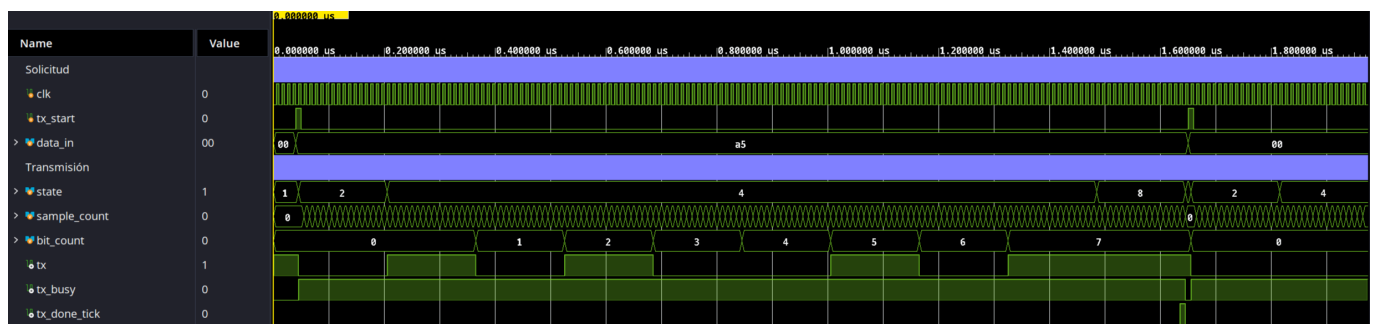


Figura 6.3: Serialización de A5 en `uart_tx_tb`.

La figura 6.4 muestra la ocupación completa del FIFO. El contador alcanza el valor cuatro y activa `full`. El intento de escribir `99` no modifica el dato ubicado en la cabeza de la cola.

Las escrituras posteriores reutilizan las posiciones liberadas, pero no alteran el orden de lectura. La secuencia observada es `11, 22, 33, 44, 55, 66, 77, 88`. Al retirar el último dato, el contador vuelve a cero y se activa `empty`.

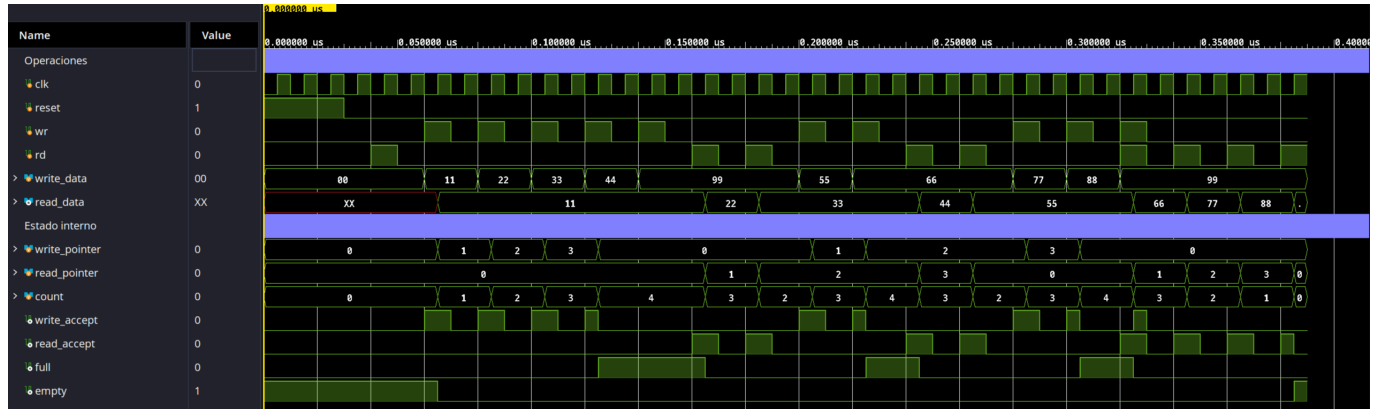


Figura 6.4: Límites, accesos simultáneos y wrap-around en `fifo_tb`.

6.2.3 Interpretación

Los cuatro bancos elementales finalizaron con el resultado PASS. Las 814 comparaciones de `alu_tb.sv` confirmaron que la integración no modificó el comportamiento de la ALU. La prueba de `baud_tick_gen_tb.sv` comprobó la periodicidad con la parametrización reducida y utilizó la misma expresión del divisor que el diseño. Este ensayo no mide la precisión del oscilador físico.

Las capturas del receptor y del transmisor muestran el recorrido completo del byte `A5`. En ellas se puede seguir el orden de los bits y el cambio de estado que ocurre durante cada parte de la trama. Los demás bytes no se incluyeron en las figuras, pero fueron comprobados automáticamente por sus respectivos bancos.

El archivo `fifo_tb.sv` verificó que los datos se retiraran en el mismo orden en que habían sido escritos. También comprobó que `full` se activara al ocupar las cuatro posiciones y que `empty` lo hiciera después de retirar el último dato.

Estas pruebas confirman el funcionamiento individual de los bloques. Todavía no comprueban que el controlador pueda recibir una solicitud completa ni que el resultado recorra todo el camino entre ambos pines UART.

6.3 Subsistemas: núcleo UART y controlador

Una vez verificados los bloques elementales, se probaron las dos partes que coordinan el intercambio. El núcleo UART reúne el receptor, el transmisor y las dos colas. El controlador interpreta los bytes recibidos y genera la respuesta de la ALU. Los bancos de este nivel permiten revisar esas funciones sin incorporar todavía el recorrido completo por las líneas serie.

6.3.1 Criterio y estímulos

El archivo `tb/uart_core_tb.sv` verifica primero el camino de recepción. Para ello envía cinco bytes consecutivos sobre RX sin retirar ningún dato. Los primeros cuatro deben permanecer almacenados en el FIFO. La llegada del quinto debe generar exactamente un overrun. Después, el banco retira los cuatro bytes y comprueba que mantengan el orden de llegada.

El mismo banco verifica luego el camino de transmisión. En este caso escribe `A5`, `00`, `FF` y `3C` en la cola correspondiente. Un decodificador implementado en el banco reconstruye las cuatro tramas observadas sobre TX y compara los bytes obtenidos.

La prueba del controlador se encuentra en `tb/alu_uart_controller_tb.sv`. Este banco entrega y retira bytes a través de la interfaz de los FIFO. De este modo puede comprobar la interpretación de la secuencia A–B–opcode sin incluir los tiempos propios de la transmisión UART.

Los casos dirigidos incluyen una suma normal, una suma con carry y otra con overflow. También verifican SRA, AND, SUB, NOR, un opcode inválido y un opcode con los bits superiores reservados. Para comprobar la espera de la salida, el banco mantiene `tx_full` activo durante cinco ciclos. Por último, deja vencer el timeout de una operación incompleta y aplica `rx_error_tick` después de recibir el operando A.

6.3.2 Resultado observado

La figura 6.5 permite seguir primero la operación `7F 01 20`. En la parte superior aparecen los tres bytes de la solicitud sobre `r_data`. Cada vez que hay un byte disponible, el controlador activa `rd_uart` para retirarlo de la cola de recepción. Las tres lecturas almacenan el operando A, el operando B y el opcode. Al mismo tiempo, el estado avanza por `WAIT_A`, `WAIT_B` y `WAIT_OPCODE`.

Después de recibir el opcode, el controlador pasa a `EXECUTE`. El pulso `capture_result` registra el resultado `80` y las banderas de la ALU. En ese momento, `result_reg` toma el valor `80` y los LED cambian a `480`. Los tres bits superiores de este patrón indican que la bandera V está activa. Como la cola de transmisión tiene espacio, `w_data` presenta el resultado y `wr_uart` genera el pulso que lo escribe en el FIFO.

La parte derecha de la captura muestra una situación diferente con la operación `0C 0A 24`. El controlador recibe los tres bytes y calcula el resultado `C0`, pero `tx_full` se encuentra en alto. Por esta razón, permanece en `WAIT_TX` y conserva la respuesta en `result_reg`. Durante esa espera, `wr_uart` permanece en bajo y el resultado no se escribe en una cola que ya está llena.

Cuando `tx_full` vuelve a cero, el controlador genera el pulso de escritura y entrega la respuesta pendiente. Recién entonces regresa a `WAIT_A` para comenzar una nueva solicitud. Esta parte de la figura muestra que una demora en la transmisión no provoca la pérdida del resultado calculado.

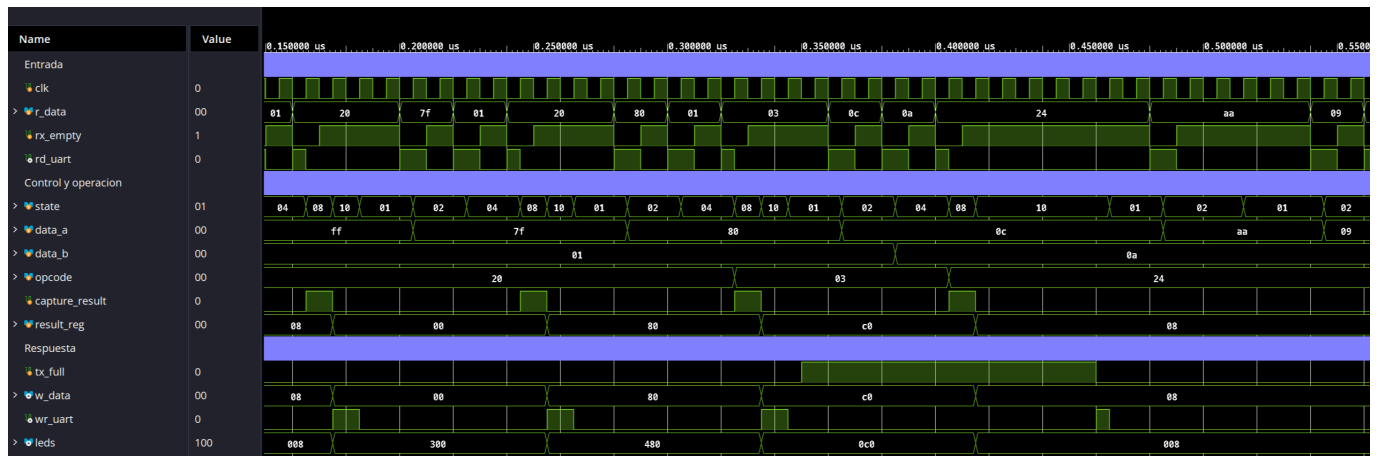


Figura 6.5: Operación y espera por capacidad en `alu_uart_controller_tb`.

XSim informó `PASS: UART core FIFOs, overrun and serialized output` al finalizar el banco del núcleo. Para el controlador informó `PASS: ALU/UART controller protocol, stalls and recovery`. Ninguno de los bancos produjo errores fatales ni alcanzó el límite de su watchdog.

6.3.3 Interpretación

El banco del núcleo confirmó que las colas conservan el orden de los bytes. La prueba de recepción completa mostró además que un dato adicional produce un error de overrun en lugar de sobrescribir el elemento más antiguo.

El banco del controlador confirmó que una respuesta permanece registrada mientras la cola de transmisión está llena. También verificó que EXECUTE separa la captura del opcode del registro del resultado. Los casos de timeout y error de recepción mostraron que una solicitud incompleta se descarta antes de aceptar una nueva.

En estas pruebas, uno de los extremos todavía es reemplazado por el propio banco. El núcleo UART no está conectado a la ALU y el controlador recibe bytes completos sin atravesar el receptor. Por lo tanto, los resultados de este nivel no sustituyen una prueba del sistema integrado.

6.4 Verificación extremo a extremo

El último nivel comprueba el funcionamiento del módulo superior. En esta prueba los estímulos ingresan como bits por `serial_rx`. La respuesta también se observa como una trama completa sobre `serial_tx`. De esta manera se incluye el mismo recorrido que utiliza una operación dentro de la FPGA.

6.4.1 Criterio y estímulos

El archivo `tb/alu_uart_top_tb.sv` instancia el sincronizador, el generador de ticks, el receptor, el transmisor, los dos FIFO, el controlador y la ALU. Para conservar las relaciones temporales y reducir el costo de la simulación, utiliza un reloj de 100 kHz, una velocidad de 2500 baud y un sobremuestreo 4x. Con estos parámetros, el divisor permanece en diez y cada bit dura cuarenta ciclos.

El banco no fuerza ninguna señal interna. Los estímulos modifican únicamente `serial_rx` y la respuesta se reconstruye a partir de `serial_tx`.

Los primeros doce casos son dirigidos. En las operaciones válidas se incluyen una suma normal, una suma con carry, una suma con overflow, SUB, SRA, SRL y NOR. También se comprueban un opcode inválido y un opcode con los bits superiores reservados.

Los tres casos restantes se concentran en la recuperación. El primero envía un byte aislado y espera que venza el timeout. El segundo introduce un bit de stop inválido y luego transmite una operación correcta. El tercero activa el reset durante una trama.

Después de los casos dirigidos, el banco aplica cien pares pseudoaleatorios para cada una de las ocho operaciones. La secuencia utiliza la semilla `32'h1a2b3c4d`. En total se ejecutan 800 operaciones aleatorias y 812 transacciones.

6.4.2 Resultado observado

La figura 6.6 muestra la primera operación, representada por los bytes `05 03 20`. Sobre RX se distinguen tres grupos de diez bits. Cada pulso `rx_done` publica uno de los bytes y hace avanzar al controlador.

Después de recibir el opcode se activa `enqueue`. La cola de transmisión recibe `08` y `serial_tx` comienza a emitir la trama de respuesta. Antes de completar la operación, los LED muestran `100`. Este valor corresponde al resultado cero con la bandera Z activa. Luego cambian a `008`, que representa el resultado ocho sin banderas activas.



Figura 6.6: Recorrido completo de `05 03 20` en `alu_uart_top_tb`.

La regresión completa se compiló y ejecutó nuevamente mediante el archivo `scripts/run_tests.sh` con XSim 2025.2. La figura 6.7 reúne las líneas finales emitidas por los ocho bancos. El registro completo de la ejecución se conserva en `reports/simulation_regression.log`.

El banco superior finalizó con `PASS: 812 complete serial ALU transactions`. Después de ejecutar los ocho bancos, el script informó `PASS: 8 testbenches completados`.

```
source alu_uart_top_tb.tcl
## set curr_wave [current_wave_config]
## if ( [string length $curr_wave] == 0 ) {
##   if ( [llength [get_objects]] > 0 ) {
##     add_wave /
##     set_property needs_save false [current_wave_config]
##   } else {
##     send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform Configuration' or type 'create_wave_config' in the TCL console."
##   }
## }
## run 0ns
INFO: [USF-XSim-96] XSim completed. Design snapshot 'alu_uart_top_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 0ns
launch_simulation: Time (s): cpu = 00:00:07 ; elapsed = 00:00:06 . Memory (MB): peak = 11896.418 ; gain = 0.000 ; free physical = 1658 ; free virtual = 13979
PASS: 812 complete serial ALU transactions
$finish called at time : 12700466 ns : File "/home/franco/college/computer_architecture/labs/tps/tp2/tb/alu_uart_top_tb.sv" Line 250
# puts ""

# puts "===== RESUMEN DE REGRESION ====="
# puts "===== RESUMEN DE REGRESION ====="
# foreach testbench $completed {
#   puts "PASS: $testbench"
# }
PASS: alu_tb
PASS: baud_tick_gen_tb
PASS: fifo_tb
PASS: uart_rx_tb
PASS: uart_tx_tb
PASS: uart_core_tb
PASS: alu_uart_controller_tb
PASS: alu_uart_top_tb
# puts "PASS: [llength $completed]/[llength $testbenches] testbenches completados con XSim"
PASS: 8/8 testbenches completados con XSim
# puts "=====
```

Figura 6.7: Resultado de la regresión completa ejecutada con XSim 2025.2.

Banco	Estado	Resultado principal
alu_tb	PASS	814 comprobaciones.
baud_tick_gen_tb	PASS	Tick cada diez ciclos.
fifo_tb	PASS	Fronteras y wrap-around.
uart_rx_tb	PASS	Tramas válidas y defectuosas.
uart_tx_tb	PASS	Framing, orden y tiempos.
uart_core_tb	PASS	FIFO, overrun y salida serie.
alu_uart_controller_tb	PASS	Protocolo y recuperación.
alu_uart_top_tb	PASS	812 transacciones completas.

Tabla 6.2: Resultado de la regresión RTL completa.

6.4.3 Interpretación

El banco superior comprobó el recorrido funcional desde el pin RX hasta el pin TX. La prueba incluyó el orden de los bits, el almacenamiento de los bytes, la ejecución de la ALU, la transmisión de la respuesta y la permanencia del resultado en los LED.

Los casos de recuperación confirmaron que el timeout, el bit de stop inválido y el reset no producen una respuesta espuria. Después de cada situación, una transacción nueva pudo completarse correctamente. Las 800 operaciones aleatorias ampliaron la cantidad de operandos ejercitados mediante una secuencia que puede repetirse con la misma semilla.

Estos resultados comprueban el comportamiento funcional del RTL, pero no representan todos los fenómenos presentes en el circuito físico. La simulación no modela la metastabilidad analógica ni el ruido de las señales. Tampoco incluye la tolerancia de los osciladores, las demoras del enlace USB ni los retardos obtenidos después del ruteo.

El análisis CDC y la verificación temporal del diseño implementado consideran parte de estos aspectos en las secciones siguientes. La comunicación con el puente USB–UART y la observación de los LED requieren una comprobación posterior sobre la placa.

Sección 7

Síntesis e implementación

7.1 Flujo sobre el dispositivo objetivo

La simulación funcional permite comprobar el comportamiento del RTL, pero no muestra cómo se utiliza el dispositivo físico. Para obtener esa información, el diseño se sintetizó y se implementó sobre la FPGA Artix-7 [xc7a35tcpg236-1](#) de la Basys 3.

Durante la síntesis, Vivado interpreta los módulos SystemVerilog y los traduce a una red formada por registros, memorias, operadores y máquinas de estado. La implementación toma esa red y asigna cada recurso a una ubicación concreta del dispositivo. En esta etapa también realiza la colocación, el ruteo y el cálculo de los retardos físicos. Una vez completadas ambas etapas, se genera el bitstream utilizado para programar la placa.

El proceso se automatizó mediante dos archivos. El script [scripts/create_project.tcl](#) crea el proyecto para el dispositivo seleccionado. También agrega las diez fuentes RTL, los ocho bancos de pruebas y el archivo de restricciones [basys3.xdc](#). Finalmente, establece [alu_uart_top](#) como módulo superior.

El script [scripts/build_bitstream.tcl](#) ejecuta la construcción del diseño. Primero reinicia las corridas anteriores y vuelve a realizar la síntesis y la implementación. Luego guarda en el directorio [reports/](#) los reportes de utilización, metodología, temporización, CDC y DRC. El mismo script genera el archivo [build/alu_uart_top.bit](#).

La creación del proyecto y la construcción completa pueden repetirse con los siguientes comandos:

```
vivado -mode batch -source scripts/create_project.tcl
vivado -mode batch -source scripts/build_bitstream.tcl
```

La construcción utilizada para este informe se realizó con Vivado 2025.2, build 6299465. La síntesis terminó sin errores, advertencias críticas ni advertencias. Durante la implementación, el ruteo finalizó con el mensaje [Router Completed Successfully](#). La figura 7.1 muestra el estado final de ambas corridas.

Name	Constraints	Status	LUT	FF	BRAM	U
✓ synth_1	constrs_1	synth_design Complete!	197	126	0	
▶ impl_1	constrs_1	Not started				

Name	Constraints	Status	WNS	TNS	WHS	THS	TPWS	Total Power	Failed Routes	LU
✓ synth_1	constrs_1	synth_design Complete!								19
✓ impl_1	constrs_1	write_bitstream Complete!	4.524	0.00	0.037	0.000	0.000	0.076	0	19

Figura 7.1: Corridas de síntesis e implementación completadas en Vivado.

El bitstream obtenido corresponde al dispositivo declarado y ocupa 326538 bytes. La figura 7.2 resume el recorrido desde las fuentes SystemVerilog hasta ese archivo. Los resultados de utilización, temporización, CDC y DRC se producen durante las etapas intermedias y se analizan por separado en las secciones siguientes.

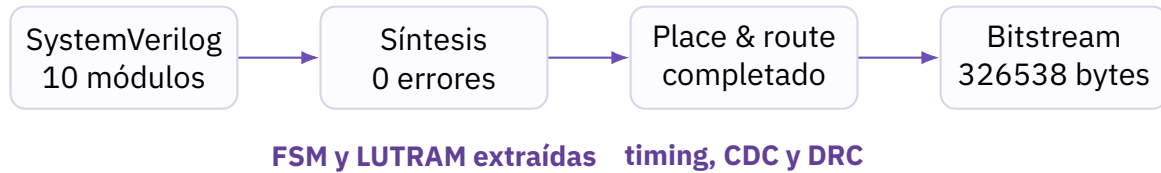


Figura 7.2: Recorrido reproducido desde RTL hasta el bitstream.

7.2 Elaboración y comprobación del RTL

Antes de traducir el diseño a los recursos de la FPGA, Vivado elabora las fuentes y construye su jerarquía lógica. Esta etapa permite comprobar que los módulos, las instancias y sus conexiones fueron interpretados de acuerdo con la descripción SystemVerilog.

La figura 7.3 muestra las fuentes incorporadas al proyecto. Allí se reconoce el archivo `rtl/alu_uart_top.sv` como nivel superior, junto con sus instancias sintetizables. También aparecen `basys3.xdc` y los ocho bancos utilizados durante la simulación.



Figura 7.3: Jerarquía de fuentes, restricciones y bancos de pruebas en Vivado.

Sobre esta misma configuración se ejecutó el análisis estático del RTL. La herramienta revisó la descripción antes de la síntesis y no encontró violaciones. El resultado se observa en la figura 7.4.

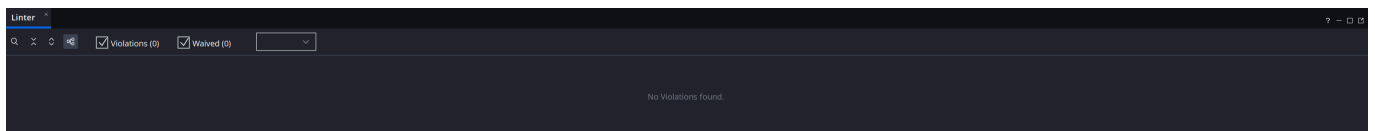


Figura 7.4: Análisis estático del RTL sin violaciones.

Los esquemáticos elaborados permiten revisar la estructura que Vivado obtuvo a partir de las fuentes. En esta vista todavía se conservan los módulos del RTL. Por lo tanto, resulta posible comparar directamente la jerarquía generada con la arquitectura presentada anteriormente.

La revisión comienza en el nivel superior. En la figura 7.5 se distinguen el acondicionador del reset, el núcleo UART y el controlador de las transacciones. Estos bloques se mantienen separados y se conectan mediante las señales definidas en `rtl/alu_uart_top.sv`.

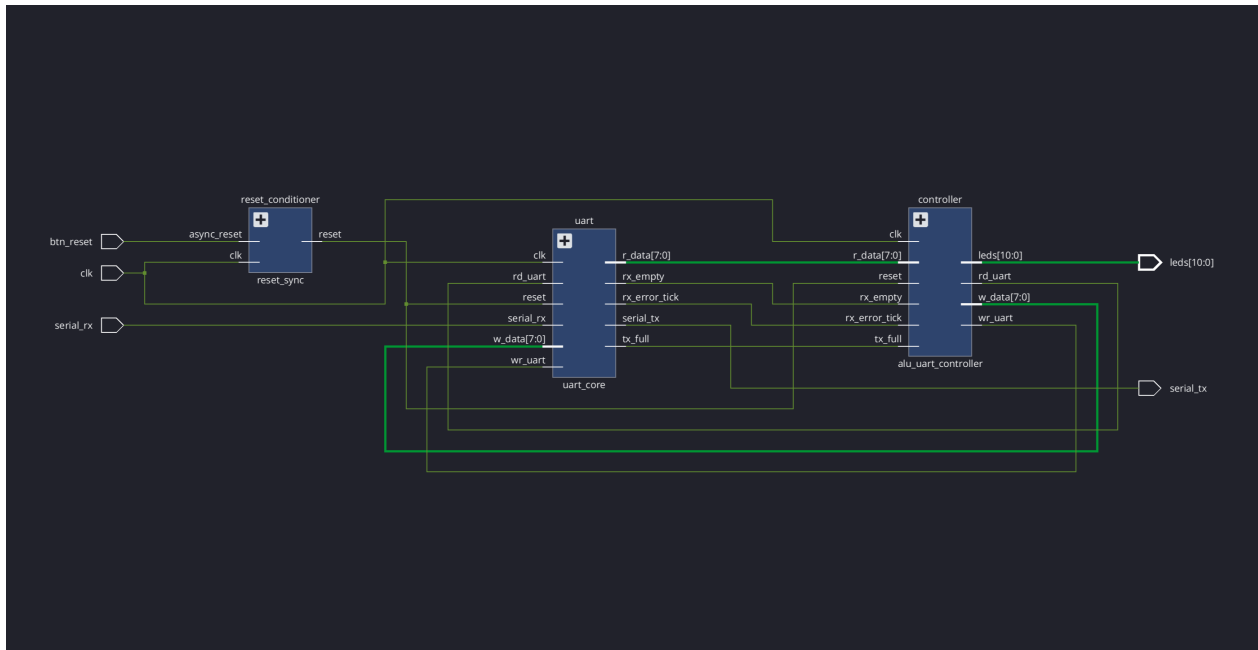


Figura 7.5: Esquemático RTL elaborado de `alu_uart_top`.

Al ingresar en el núcleo UART se observan los dos caminos de la comunicación. La figura 7.6 muestra el receptor con su FIFO y el transmisor con la cola de salida. Ambos caminos utilizan el mismo generador de muestreo. Esta organización coincide con la descripción del archivo `rtl/uart_core.sv`.

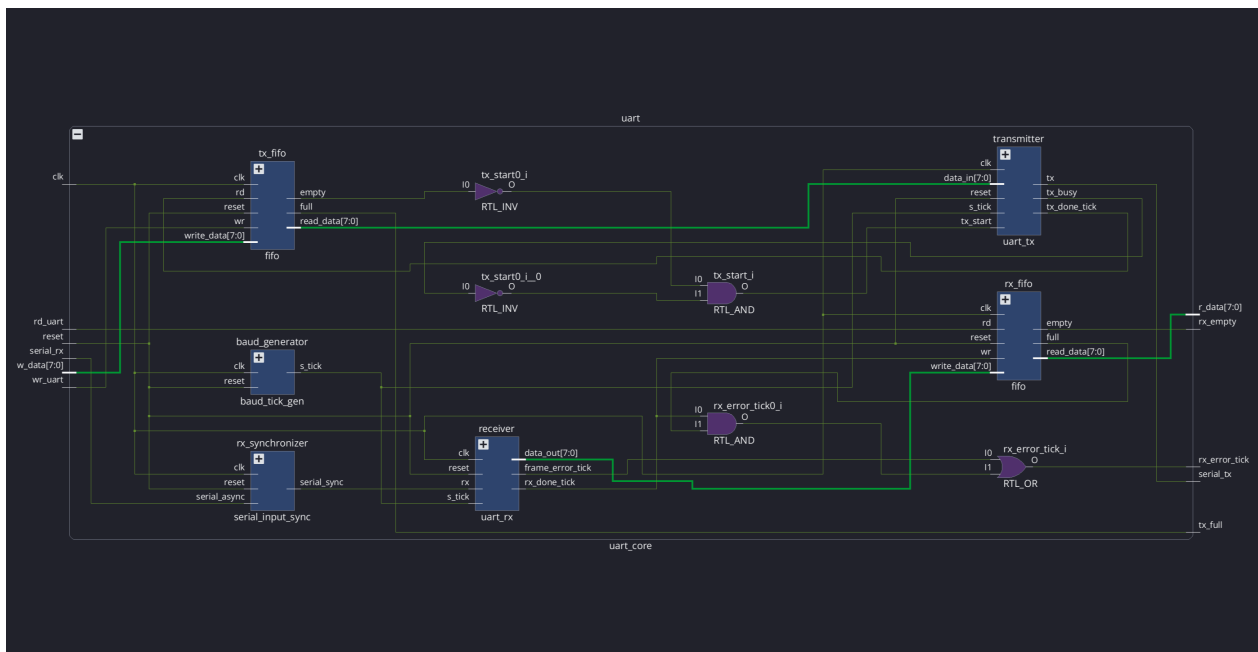


Figura 7.6: Esquemático RTL elaborado de `uart_core`.

Los circuitos que preparan las señales de entrada se revisaron por separado. La figura 7.7 reúne el acondicionador del reset, el sincronizador de `serial_rx` y el generador de `s_tick`. Los esquemáticos confirman que estas funciones permanecen en módulos independientes antes de conectarse con el resto del sistema.

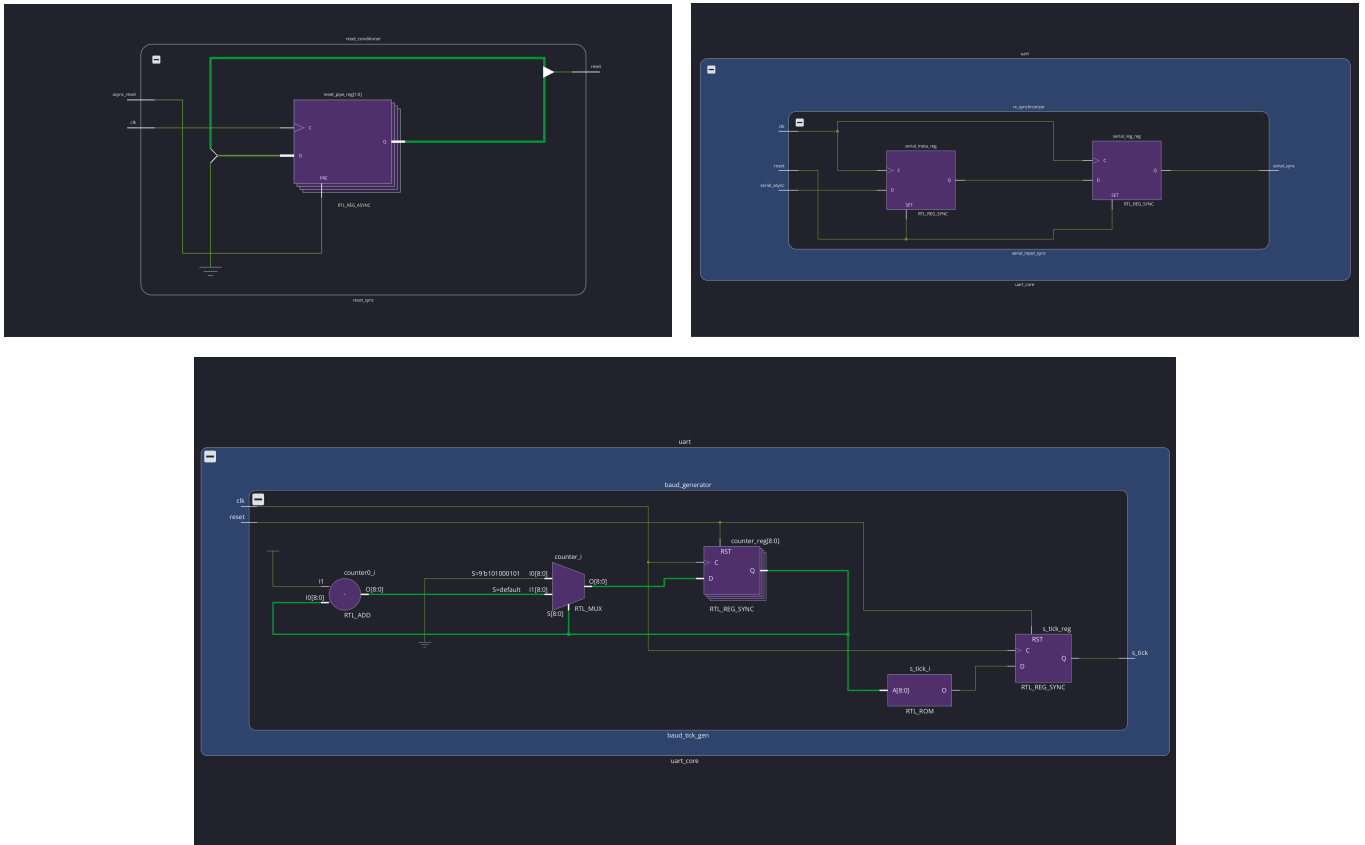


Figura 7.7: Esquemáticos RTL del acondicionamiento de reset, el sincronizador de entrada y el generador de muestreo.

La figura 7.8 presenta los dos módulos que forman y reconstruyen las tramas. El esquema de `uart_rx` corresponde al camino que convierte la entrada serie en un byte. El de `uart_tx` muestra el recorrido inverso desde un byte hasta la salida serie.

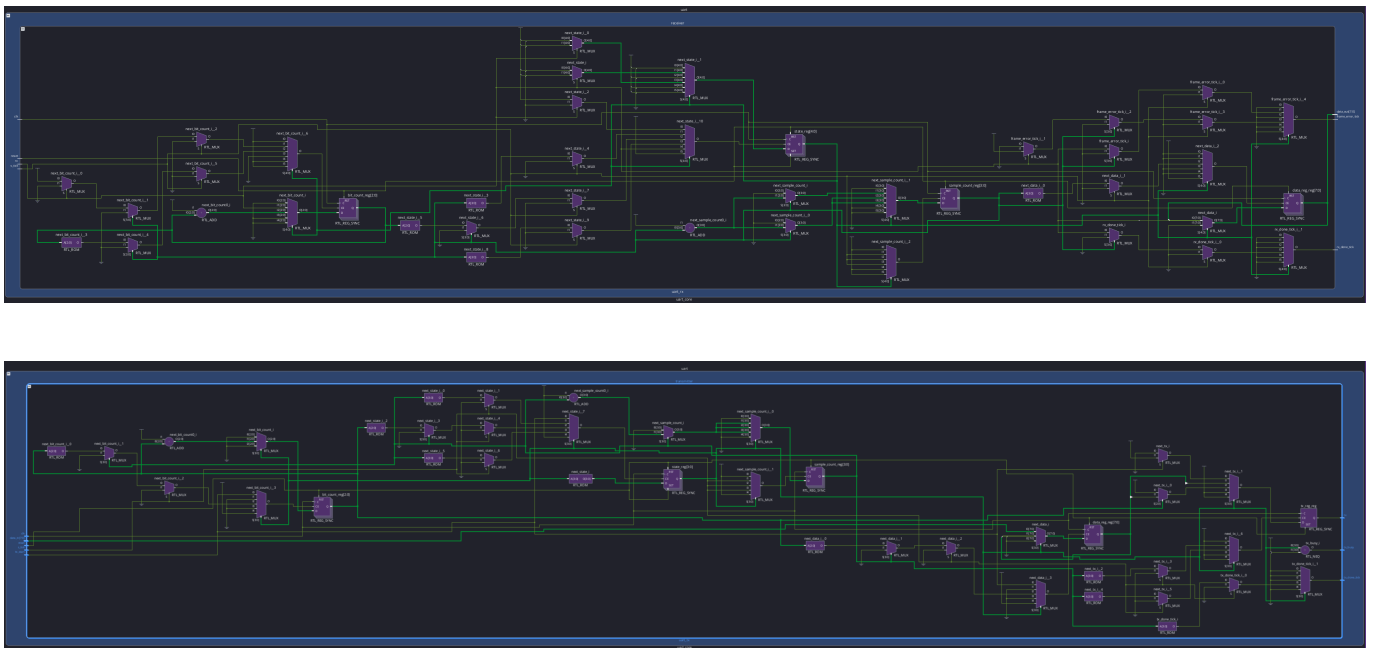


Figura 7.8: Esquemáticos RTL del receptor y del transmisor UART.

Entre los módulos UART y el controlador se encuentran las dos instancias de `fifo`. La figura 7.9 permite comparar la cola de recepción con la de transmisión. Ambas conservan la misma estructura, aunque cada una almacena los bytes de un sentido diferente del enlace.

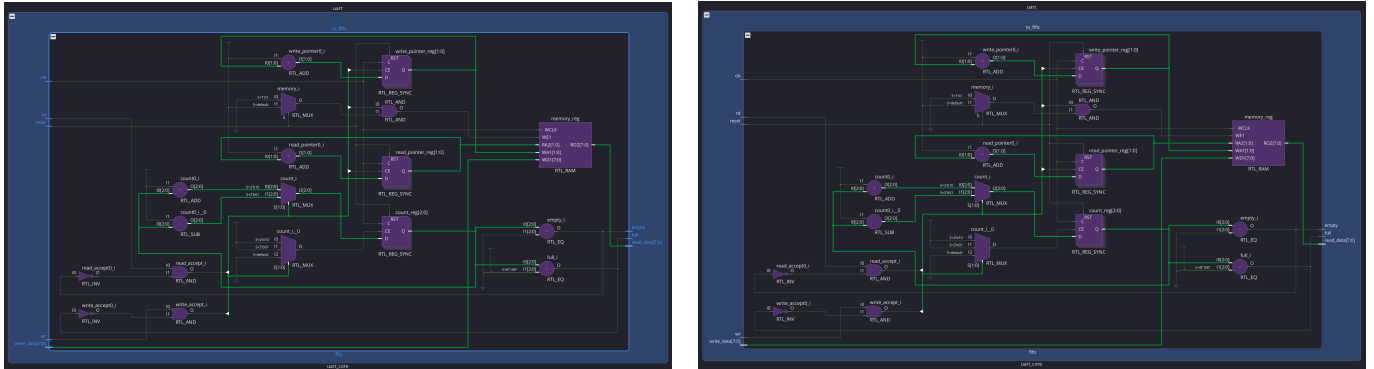


Figura 7.9: Esquemáticos RTL de las instancias FIFO de recepción y transmisión.

Una vez reconstruidos los bytes, el controlador debe interpretar la solicitud y coordinar la respuesta. La figura 7.10 muestra la estructura elaborada a partir del archivo `rtl/alu_uart_controller.sv`. En ella se mantienen diferenciados los registros de la transacción y la lógica que controla su avance.

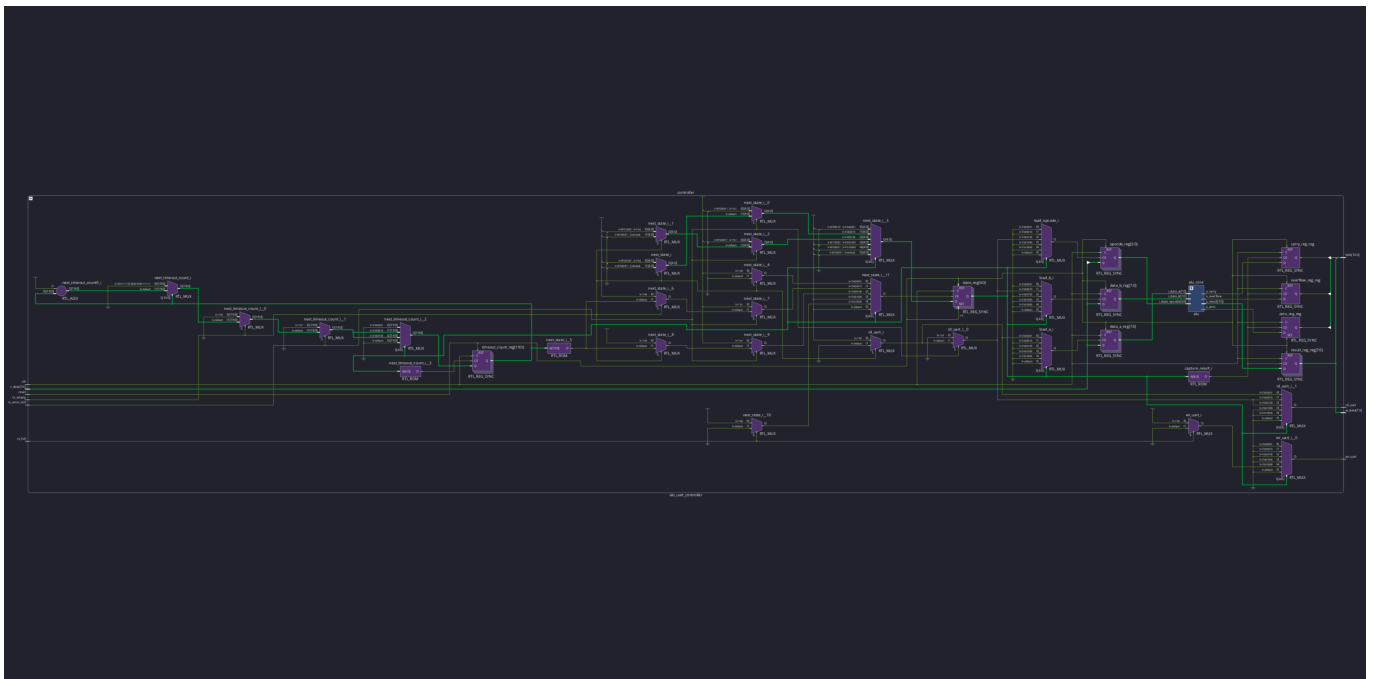


Figura 7.10: Esquemático RTL elaborado del controlador de transacciones.

La última parte del camino corresponde al cálculo. El esquemático de la figura 7.11 muestra la ALU combinacional elaborada desde `rtl/alu.sv`. La ausencia de registros dentro de este bloque coincide con la decisión de almacenar el resultado y las banderas en el controlador.

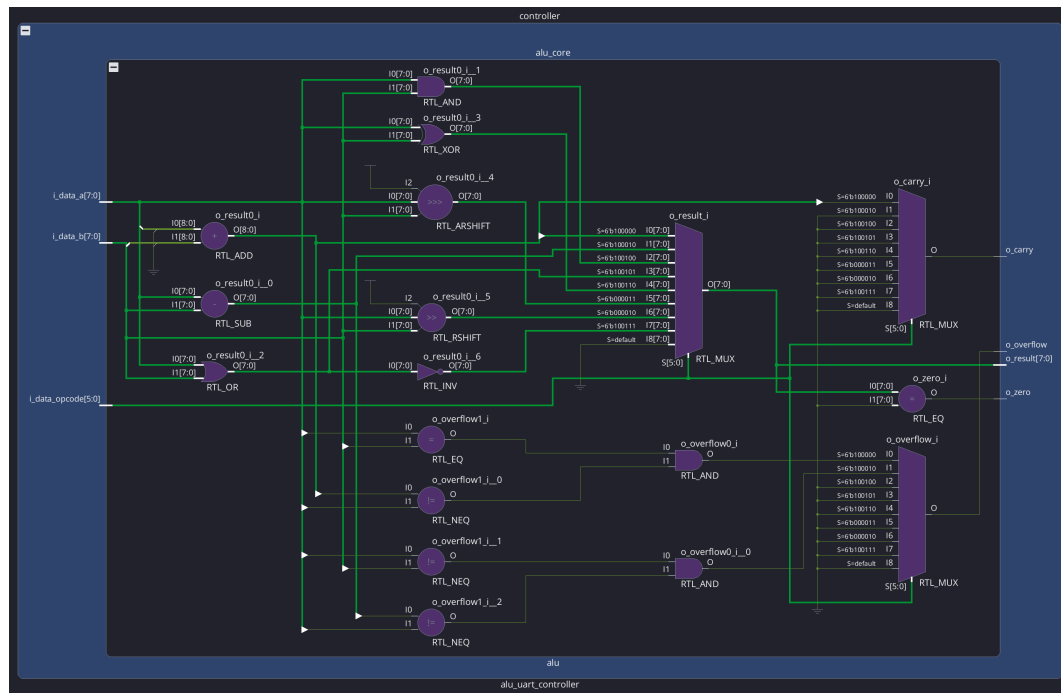


Figura 7.11: Esquemático RTL elaborado de la ALU combinacional.

En conjunto, estos esquemáticos muestran que la elaboración conservó la separación prevista entre la comunicación UART, el almacenamiento, el control de la transacción y el cálculo.

7.3 Del esquema RTL a los recursos físicos

Después de comprobar la jerarquía elaborada, se analizó la forma en que Vivado la convirtió en recursos del Artix-7. Esta comparación permite determinar si los registros, las memorias y la lógica combinacional fueron inferidos de acuerdo con la arquitectura.

Los contadores de sobremuestreo, los índices de bits, el contador del timeout y los contadores de ocupación necesitan conservar un valor entre ciclos. Por esta razón aparecen como flip-flops acompañados por las LUT que calculan su próximo valor.

Las tres máquinas de estado utilizan en total catorce bits. Cinco corresponden al receptor, cuatro al transmisor y cinco al controlador. Vivado implementa estos bits con registros y utiliza LUT para determinar las transiciones. Los catorce bits no representan estados activos al mismo tiempo. Durante el funcionamiento normal permanece activo un solo bit de cada máquina.

Cada FIFO contiene un arreglo de cuatro palabras de ocho bits. Debido a su tamaño, no resulta conveniente utilizar un bloque completo de memoria. Vivado implementa ambos arreglos como memoria distribuida en LUTRAM. Esto explica las 12 LUT utilizadas como memoria después del ruteo y la ausencia de bloques RAM.

La ALU se implementa mediante LUT y cadenas de carry para las operaciones de suma y resta. No requiere bloques DSP. Los registros de los operandos, del opcode, del resultado y de las banderas también forman parte de los 134 flip-flops informados. A esos registros se suman los utilizados para los desplazamientos y los distintos contadores.

La conexión con la placa utiliza tres buffers de entrada. Estos corresponden al reloj, el reset y la línea RX. También utiliza doce buffers de salida para TX y los once LED. En total se ocupan 15 IOB.

El reloj que ingresa por el pin W5 utiliza un único BUFG. El generador de baud no agrega otro buffer global porque `s_tick` funciona como una habilitación síncrona y no como un segundo reloj. Si la implementación hubiera incorporado una BRAM, un DSP o un segundo BUFG, habría sido necesario revisar la inferencia realizada por la herramienta.

Los esquemáticos implementados permiten observar esta traducción. A diferencia de la vista RTL, ya no muestran solamente los módulos descritos en SystemVerilog. En ellos aparecen los registros, las LUT, las cadenas de carry y las conexiones adaptadas al dispositivo después de la colocación y el ruteo.

La figura 7.12 presenta la vista general del nivel superior ya implementado.

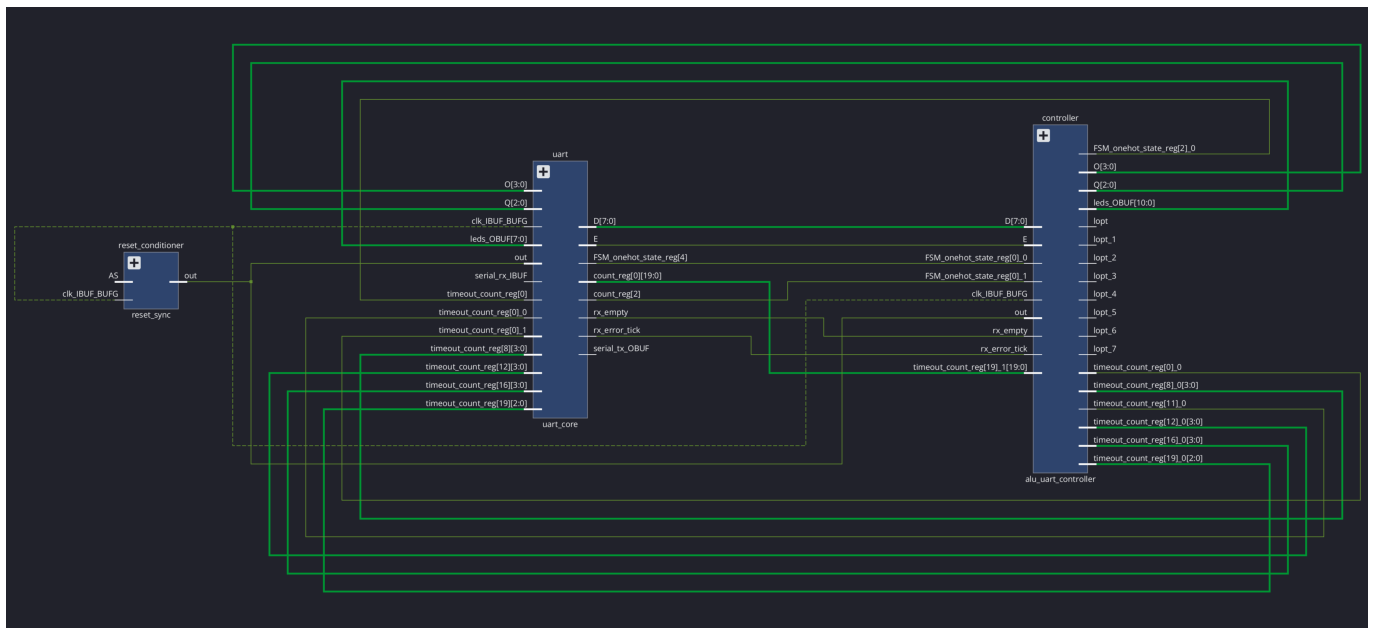


Figura 7.12: Esquemático implementado del nivel superior.

Al abrir el núcleo UART se obtiene el esquema de la figura 7.13. En esta vista se observan los recursos que forman los dos caminos de comunicación y sus conexiones internas.

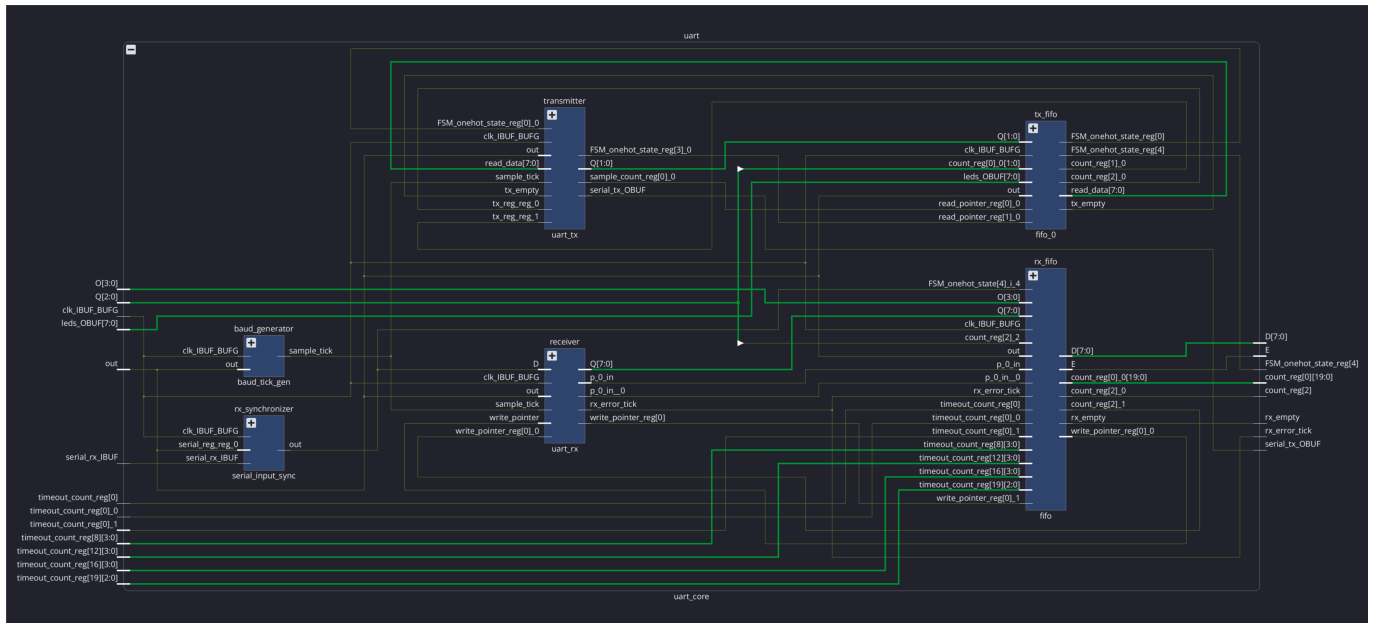


Figura 7.13: Esquemático implementado del núcleo UART.

La figura 7.14 muestra la implementación del controlador. Los registros que conservan la transacción y las LUT que determinan las transiciones reemplazan la representación de mayor nivel observada durante la elaboración.

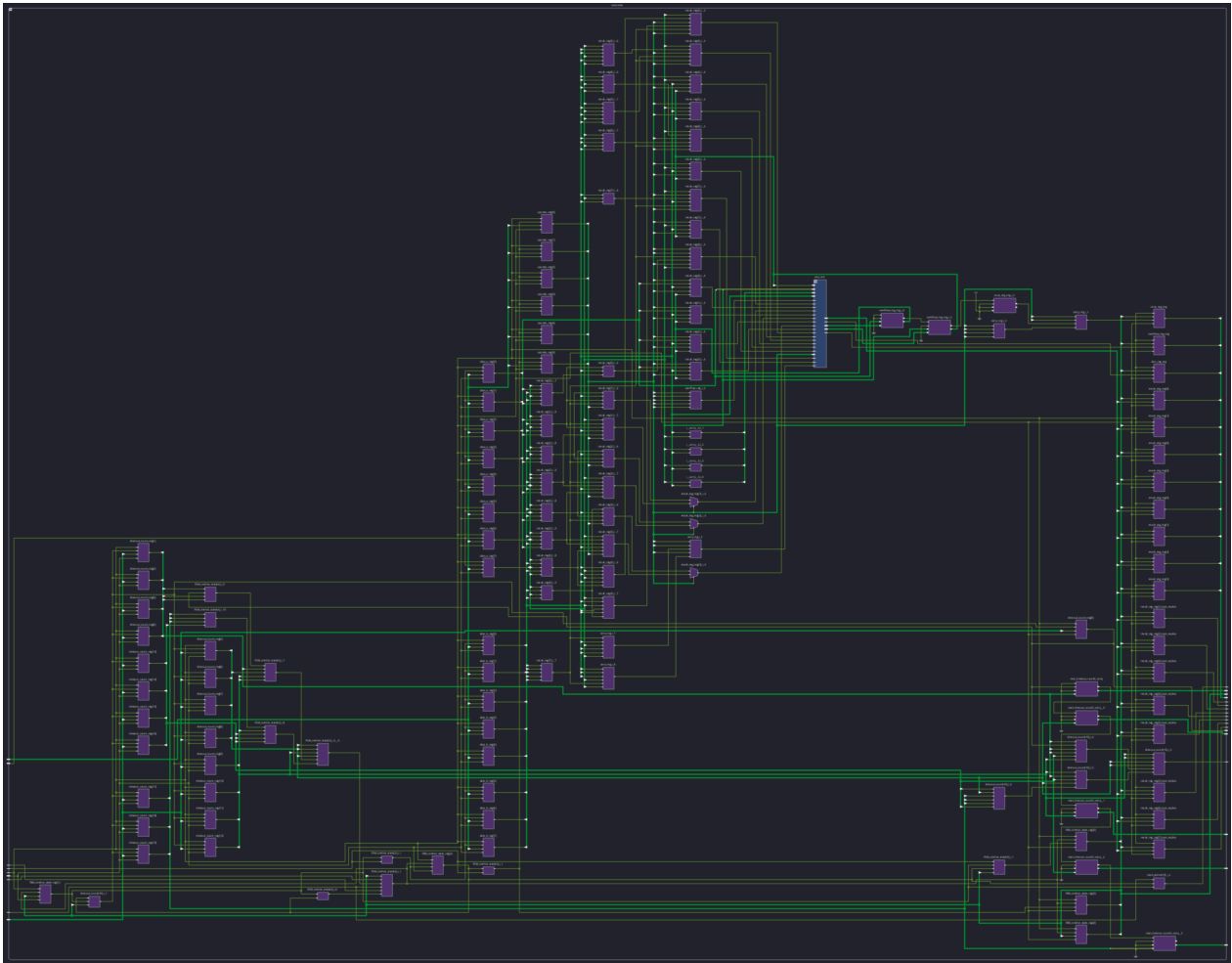


Figura 7.14: Esquemático implementado del controlador de transacciones.

Además de los esquemáticos, Vivado permite observar la ubicación de los recursos sobre el dispositivo. La figura 7.15 muestra que la lógica ocupa una parte reducida del Artix-7. También permite comprobar que los pines externos se encuentran en los bancos indicados por las restricciones de la Basys 3.

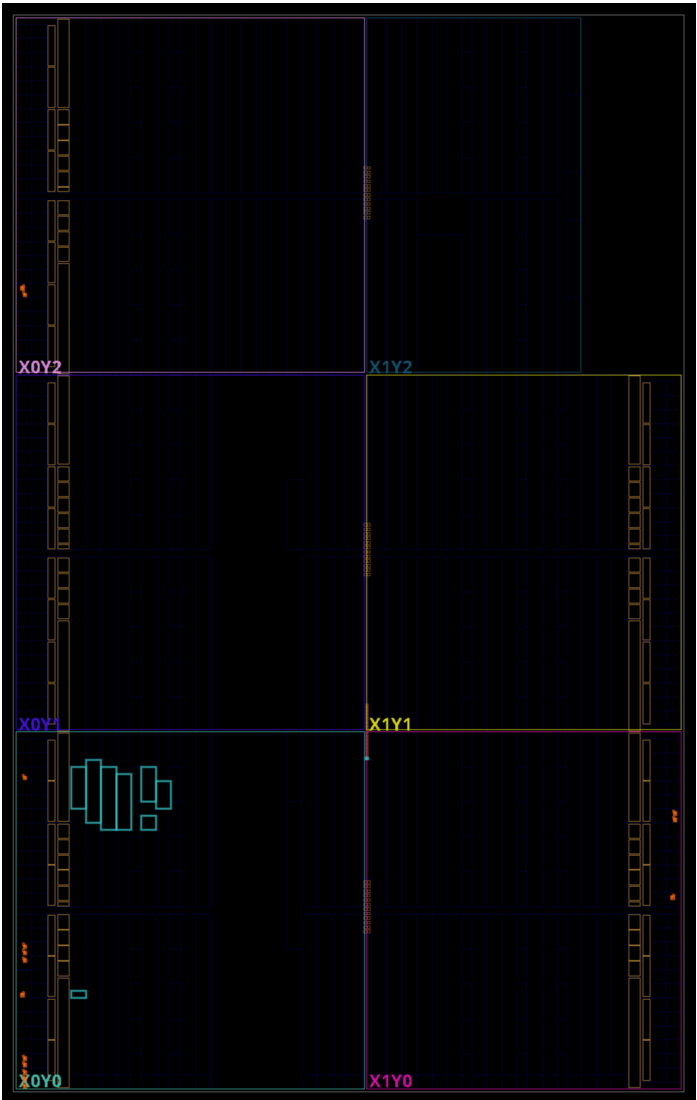


Figura 7.15: Vista del dispositivo después de place and route.

La tabla 7.1 reúne la correspondencia entre los elementos principales del RTL y los recursos observados después de la implementación.

Elemento RTL	Recurso esperado	Observado
FSM one-hot	Flip-flops y LUT de transición	14 bits de estado dentro de 134 FF.
Dos FIFO 4 × 8	Memoria distribuida	12 LUTRAM post-route.
Suma y resta ALU	LUT y carry chain	LUT/MUXF/CARRY4, 0 DSP.
Registros y contadores	Flip-flops	134 FF, 0 latches.
Reloj de placa	Buffer global	1 BUFG.
Puertos físicos	IBUF/OBUF e IOB	15 IOB asignados.

Tabla 7.1: Correspondencia entre la descripción y los recursos implementados.

7.4 Utilización de recursos

La correspondencia anterior explica qué tipo de recurso se utilizó. Para conocer cuánto espacio ocupa el diseño, se revisaron los reportes `reports/synthesis_utilization_summary.rpt` y `reports/implementation_utilization_summary.rpt`. El primero corresponde a la salida de la síntesis. El segundo contiene los valores obtenidos después de la colocación y el ruteo.

La figura 7.16 presenta el resumen posterior al ruteo. La figura 7.17 distribuye esos mismos recursos entre las instancias del diseño.

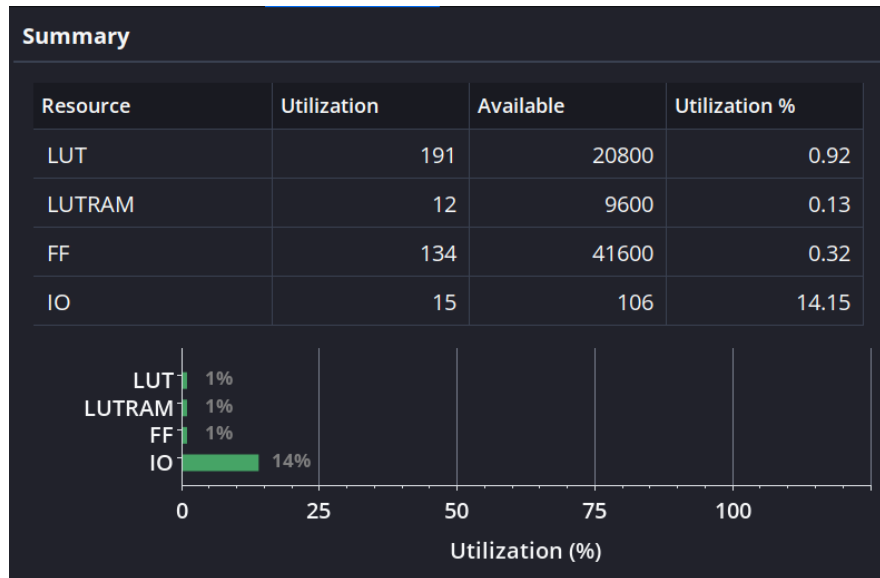


Figura 7.16: Resumen de utilización posterior al ruteo.

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	Slice (8150)	LUT as Logic (20800)	LUT as Memory (9600)	Bonded IOB (106)	BUFGCTRL (32)
alu_uart_top	191	134	11	66	179	12	15	1
controller (alu_uart_controller)	88	66	11	37	88	0	0	0
uart (uart_core)	103	66	0	40	91	12	0	0

Figura 7.17: Distribución jerárquica de los recursos implementados.

Para observar los cambios introducidos durante la implementación, la tabla 7.2 compara los valores de síntesis con los valores posteriores al ruteo.

Etapas	LUT	LUT lógica	LUTRAM	FF
Síntesis	197	181	16	126
Post-route	191	179	12	134
Disponible	20800	20800	9600	41600
Uso post-route	0,92 %	0,86 %	0,13 %	0,32 %

Tabla 7.2: Utilización obtenida antes y después de la implementación.

La cantidad total de LUT disminuye de 197 a 191. Dentro de ese total, las LUT destinadas a lógica pasan de 181 a 179 y las utilizadas como memoria se reducen de 16 a 12. Estas variaciones se producen durante las optimizaciones físicas de la implementación.

La cantidad de flip-flops aumenta de 126 a 134. Los ocho registros adicionales aparecen al adaptar la red sintetizada a los recursos del dispositivo. Tanto la reducción de LUT como el aumento de registros son pequeños frente a la capacidad disponible.

El diseño final utiliza 66 slices, 15 de los 106 IOB y uno de los 32 BUFG. No utiliza BRAM ni DSP. La ocupación de LUT y flip-flops permanece por debajo del uno por ciento. Estos valores muestran que las dos colas y las tres máquinas de estado poseen un costo reducido dentro del Artix-7. La ampliación del protocolo queda fuera del alcance de este trabajo.

Sección 8

Timing y limitaciones

8.1 Qué se comprueba

Después de ubicar y rutear el diseño, es necesario comprobar que la lógica pueda funcionar con el reloj de la Basys 3. El análisis temporal estudia los caminos que conectan los registros. Para cada uno determina si el dato producido por un registro llega a tiempo al siguiente y permanece estable durante el intervalo necesario.

La comprobación de setup considera el tiempo anterior al flanco de captura. El dato debe llegar y estabilizarse antes de ese flanco. La comprobación de hold analiza el intervalo posterior y verifica que el dato no cambie demasiado pronto. El análisis de ancho de pulso comprueba que los niveles alto y bajo del reloj duren el tiempo requerido por el dispositivo.

Cada camino termina en un punto de captura que el reporte denomina *endpoint*. El margen indica cuánto tiempo queda entre la llegada del dato y el límite correspondiente a ese punto.

Vivado informa el resultado de setup mediante WNS y TNS. El WNS corresponde al menor margen encontrado entre todos los caminos. El TNS suma únicamente los márgenes negativos.

Para hold, el WHS representa el menor margen y el THS reúne los márgenes negativos. En el análisis del ancho de pulso, el WPWS indica el peor margen y el TPWS suma las violaciones.

Un valor positivo de WNS o WHS indica que incluso el peor camino analizado cumple con la restricción. Cuando TNS o THS valen cero, no existen endpoints con margen negativo. Esto no significa que los caminos tengan un retardo nulo. Significa que ninguno supera el límite establecido.

El reloj se define en el archivo `basys3.xdc`. La restricción crea `sys_clk` con un período de 10 ns, que corresponde a una frecuencia de 100 MHz. A partir de esta definición, Vivado analiza los caminos internos que comienzan y terminan en registros controlados por ese reloj.

No todas las señales externas pueden analizarse con la misma relación temporal. Las entradas `btn_reset` y `serial_rx` pueden cambiar sin estar sincronizadas con `sys_clk`. Por eso, el archivo `basys3.xdc` declara una excepción desde ambos puertos hasta la primera etapa de sus sincronizadores mediante `set_false_path`.

Las doce salidas tampoco poseen una relación temporal con un reloj externo. Una corresponde a `serial_tx` y las otras once controlan los LED. Como no existe un reloj de captura externo asociado con `sys_clk`, sus caminos hasta los pines también se declaran con `set_false_path`.

Además de calcular los márgenes, el reporte revisa que la definición temporal no deje partes internas del diseño sin analizar. El resultado no presenta registros sin reloj, relojes constantes, lazos combinacionales ni endpoints internos sin restricción.

El análisis temporal se limita a estas propiedades. No determina si el protocolo UART interpreta correctamente una solicitud ni si el enlace USB funciona sobre la placa. El comportamiento del protocolo se comprobó mediante la simulación funcional. La conexión física se evalúa en la validación sobre la Basys 3.

8.2 Resultado posterior al ruteo

Una vez definidas las restricciones, el análisis se ejecutó sobre el diseño completamente ruteado. El archivo `reports/timing_summary.rpt` contiene los resultados obtenidos para el dispositivo y la velocidad seleccionados.

8.2.1 Márgenes temporales

El reporte incluye 458 endpoints para las comprobaciones de setup y hold. También analiza 159 endpoints para el ancho de pulso. La figura 8.1 muestra el resumen generado por Vivado. Todos los márgenes son no negativos y las sumas de las violaciones permanecen en cero.

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	4.524 ns	Worst Hold Slack (WHS)	0.037 ns	Worst Pulse Width Slack (WPWS)	3.750 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	458	Total Number of Endpoints	458	Total Number of Endpoints	159
All user specified timing constraints are met.					

Figura 8.1: Resumen temporal del diseño posterior al ruteo.

La tabla 8.1 reúne los valores correspondientes a los tres tipos de análisis.

Análisis	Peor margen	Suma negativa	Endpoints
Setup	WNS = 4,524 ns	TNS = 0 ns	458
Hold	WHS = 0,037 ns	THS = 0 ns	458
Ancho de pulso	WPWS = 3,750 ns	TPWS = 0 ns	159

Tabla 8.1: Márgenes reportados después del ruteo.

El peor camino de setup conserva un margen de 4,524 ns dentro del período de 10 ns. Esto significa que el dato llega con más de cuatro nanosegundos disponibles antes del límite de captura.

El margen de hold es menor y alcanza 0,037 ns. Aun así, permanece por encima de cero dentro del modelo temporal del dispositivo. Por esta razón, el THS es igual a cero y no se informan endpoints con una violación de hold. El análisis del ancho de pulso también cumple, con un WPWS de 3,750 ns y un TPWS igual a cero.

Estos resultados permiten afirmar que el diseño implementado satisface las restricciones declaradas para un reloj de 100 MHz.

8.2.2 Cruces asíncronos

El análisis de los caminos síncronos no alcanza para evaluar las señales que ingresan sin relación con el reloj interno. Por ese motivo también se realizó un análisis CDC, denominado *Clock Domain*

Crossing. Su resultado se guarda en `reports/cdc.rpt`. Este análisis comprueba que los cruces entre dominios de reloj atraviesen las estructuras de sincronización previstas.

El reporte identifica exactamente dos cruces. El primero corresponde a la liberación del reset y aparece clasificado como CDC-9. El segundo corresponde a la entrada `serial_rx` y aparece como CDC-3. Ambos sincronizadores tienen una profundidad de dos etapas y poseen la propiedad `ASYNC_REG`. Además, los caminos desde los puertos externos están cubiertos por las excepciones declaradas en `basys3.xdc`. El reporte identifica ambas excepciones como `False Path`.

La figura 8.2 muestra ambos cruces y permite comprobar su clasificación, su profundidad y la excepción aplicada.

Severity	ID	Description	Depth	Exception	Source (From)	Destination (To)	Category
Info	CDC-9	Asynchronous reset synchronized with ASYNC_REG property	2	False Path	btn_reset	reset_conditio...pipe_reg[0]/PRE	Safe
Info	CDC-3	1-bit synchronized with ASYNC_REG property	2	False Path	serial_rx	uart/rx_synchro...rial_meta_reg/D	Safe

Figura 8.2: Cruces de dominio identificados por el análisis CDC.

8.2.3 DRC y metodología

Después de revisar la temporización y los cruces asíncronos, se ejecutó el DRC, o *Design Rule Check*. Esta comprobación revisa las reglas del dispositivo sobre el diseño completamente ruteado. El archivo `reports/drc.rpt` informa `Checks found: 0`. Por lo tanto, no se encontró ninguna violación.

La figura 8.3 presenta el resumen de esta comprobación.

```
puts [report_drc -return_string]
Command: report_drc -return_string
INFO: [IP_Flow 19-1839] IP Catalog is up to date.
INFO: [DRC 23-27] Running DRC with 8 threads
report_drc completed successfully
Copyright 1986-2022 Xilinx, Inc. All Rights Reserved. Copyright 2022-2025 Advanced Micro Devices, Inc. All Rights Reserved.

-----
| Tool Version : Vivado v.2025.2 (lin64) Build 6299465 Fri Nov 14 12:34:56 MST 2025
| Date        : Fri Sep 11 15:04:08 2026
| Host       : archlinux running 64-bit unknown
| Command    : report_drc -return_string
| Design     : alu_uart_top
| Device     : xc7s35tcbg236-1
| Speed File : -1
| Design State : Fully Routed
-----

Report DRC

Table of Contents
-----
1. REPORT SUMMARY
2. REPORT DETAILS

1. REPORT SUMMARY
-----
Netlist: netlist
Floorplan: impl_1
Design limits: <entire design considered>
Ruledeck: default
Max checks: <unlimited>
Checks found: 0

+-----+-----+-----+-----+
| Rule | Severity | Description | Checks |
+-----+-----+-----+-----+

2. REPORT DETAILS
-----
```

Figura 8.3: DRC del diseño completamente ruteado, sin violaciones.

Los reportes de metodología revisan otro aspecto del proceso. En este caso, Vivado busca construcciones o decisiones que no respeten sus recomendaciones para la síntesis y la implementación. El archivo `reports/synthesis_methodology.rpt` no contiene hallazgos después de la síntesis.

El archivo `reports/implementation_methodology.rpt` tampoco presenta hallazgos después de completar la implementación.

Las figuras 8.4 y 8.5 muestran los resúmenes de ambos reportes.

Las capturas permiten observar los resultados principales, pero los reportes conservan el detalle completo de cada análisis. Los archivos utilizados son `reports/timing_summary.rpt`, `reports/cdc.rpt`, `reports/drc.rpt`, `reports/synthesis_methodology.rpt` y `reports/implementation_methodology.rpt`.

```

open_run synth_1; puts [report_methodology -return_string]
Design is defaulting to impl run constrset: constrs_1
Design is defaulting to synth run part: xc7a35tcpg236-1
Netlist sorting complete. Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 12881.859 ; gain = 0.000 ; free physical = 3574 ; free virtual = 12642
INFO: [Netlist 29-17] Analyzing 27 Unisim elements for replacement
INFO: [Netlist 29-28] Unisim Transformation completed in 0 CPU seconds
INFO: [Project 1-479] Netlist was created with Vivado 2025.2
INFO: [Project 1-570] Preparing netlist for logic optimization
Parsing XDC File [/home/franco/college/computer_architecture/labs/tps/tp2/basys3.xdc]
Finished Parsing XDC File [/home/franco/college/computer_architecture/labs/tps/tp2/basys3.xdc]
INFO: [Opt 31-138] Pushed 0 Inverter(s) to 0 load pin(s).
Netlist sorting complete. Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 12881.859 ; gain = 0.000 ; free physical = 3569 ; free virtual = 12638
INFO: [Project 1-111] Unisim Transformation Summary:
  A total of 6 instances were transformed:
  RAM32M => RAM32M (RAM032(x0), RAM522(x2)): 2 instances
  RAM32X1D => RAM32X1D (RAM032(x2)): 4 instances

Command: report_methodology -return_string
INFO: [Timing 38-35] Done setting XDC timing constraints.
INFO: [Timing 38-35] Done setting XDC timing constraints.
INFO: [DRC 23-133] Running Methodology with 8 threads
report_methodology completed successfully
Copyright 1986-2022 Xilinx, Inc. All Rights Reserved. Copyright 2022-2025 Advanced Micro Devices, Inc. All Rights Reserved.
-----
| Tool Version : Vivado v.2025.2 (lin64) Build 6299465 Fri Nov 14 12:34:56 MST 2025
| Date        : Fri Sep 11 15:06:06 2026
| Host       : archlinux running 64-bit unknown
| Command    : report_methodology -return_string
| Design     : alu_uart_top
| Device     : xc7a35tcpg236-1
| Speed File : -1
| Design State : Synthesized
-----

Report Methodology

Table of Contents
-----
1. REPORT SUMMARY
2. REPORT DETAILS

1. REPORT SUMMARY
-----
Netlist: netlist
Floorplan: design_1
Design limits: <entire design considered>
Max checks: <unlimited>
Checks Found: 0

+-----+
| Rule | Severity | Description | Checks |
+-----+
+-----+
+-----+

2. REPORT DETAILS
-----

```

Figura 8.4: Reporte de metodología posterior a síntesis, sin hallazgos.

```

puts [report_methodology -return_string]
Command: report_methodology -return_string
INFO: [Timing 38-35] Done setting XDC timing constraints.
INFO: [DRC 23-133] Running Methodology with 8 threads
report_methodology completed successfully
Copyright 1986-2022 Xilinx, Inc. All Rights Reserved. Copyright 2022-2025 Advanced Micro Devices, Inc. All Rights Reserved.
-----
| Tool Version : Vivado v.2025.2 (lin64) Build 6299465 Fri Nov 14 12:34:56 MST 2025
| Date        : Fri Sep 11 15:05:10 2026
| Host       : archlinux running 64-bit unknown
| Command    : report_methodology -return_string
| Design     : alu_uart_top
| Device     : xc7a35tcpg236-1
| Speed File : -1
| Design State : Fully Routed
-----

Report Methodology

Table of Contents
-----
1. REPORT SUMMARY
2. REPORT DETAILS

1. REPORT SUMMARY
-----
Netlist: netlist
Floorplan: impl_1
Design limits: <entire design considered>
Max checks: <unlimited>
Checks Found: 0

+-----+
| Rule | Severity | Description | Checks |
+-----+
+-----+
+-----+

2. REPORT DETAILS
-----

```

Figura 8.5: Reporte de metodología posterior a implementación, sin hallazgos.

Sección 9

Validación física

9.1 Objetivo y montaje

La validación física comprobó el funcionamiento del sistema completo sobre la Basys 3. A diferencia de la simulación y del análisis posterior al ruteo, este ensayo incluyó el puente USB–UART, el oscilador de la placa y los LED utilizados como salida. El recorrido observado comenzó en la interfaz gráfica de la computadora, atravesó el enlace serie y la ALU implementada en la FPGA, y terminó tanto en la respuesta UART como en los LEDs de la placa.

Para realizar las pruebas, la Basys 3 se conectó a la computadora mediante su puerto Micro-USB de programación. Esta conexión permitió alimentar la placa, cargar el diseño en la FPGA y utilizar el enlace UART desde la misma computadora.

9.2 Programación y conexión

El proyecto se abrió en Vivado y se estableció la conexión desde *Open Hardware Manager* mediante *Open Target* → *Auto Connect*. Hardware Manager detectó el dispositivo `xc7a35t_0`, sobre el cual se cargó `build/alu_uart_top.bit` con *Program Device*. La figura 9.1 muestra el dispositivo en estado *Programmed*.

Name	Status
localhost (1)	Connected
xilinx_tcf/Digilent/210183AA0D	Open
xc7a35t_0 (1)	Programmed
XADC (System Monitor)	

Figura 9.1: Basys 3 detectada y programada desde Vivado Hardware Manager.

Después de la programación se presionó y se soltó BTNC para llevar todos los registros a su estado inicial.

Para operar el sistema se inició la aplicación gráfica desarrollada en Python:

```
tools/.venv/bin/python tools/uart_alu_gui.py
```

La aplicación enumeró los puertos disponibles y reconoció el puente de Digilent como `/dev/ttyUSB1`. Se seleccionaron 19 200 baud. Al presionar *Conectar*, el indicador cambió de *Sin conexión* a *Conectado*, como se observa en la figura 9.2. El selector de baud configura el extremo de la computadora; no modifica el bitstream, por lo que debe coincidir con el valor implementado en la FPGA.

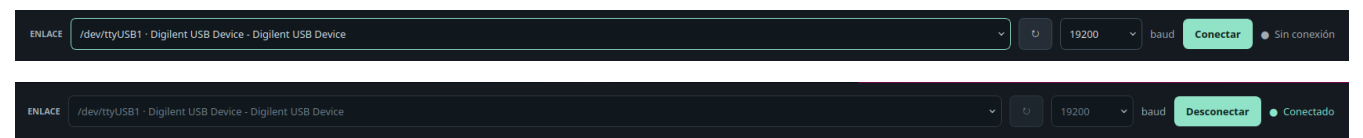


Figura 9.2: Selección del puente USB–UART y conexión de la GUI a `/dev/ttyUSB1` a 19 200 baud.

9.3 Pruebas realizadas

9.3.1 Casos dirigidos

La primera ejecución aplicó los trece casos dirigidos provistos por la vista *Pruebas*. El conjunto cubre las ocho operaciones, resultados nulos, carry, overflow y un opcode inválido. Para cada fila, la aplicación calculó el valor esperado con el modelo de referencia, envió los tres bytes A–B–opcode y comparó el único byte recibido desde la placa.

La ejecución finalizó con 13 operaciones completadas, 13 correctas y ninguna diferencia. La figura 9.3 conserva además los operandos, el opcode, los valores esperado y recibido, y el tiempo de cada transacción.

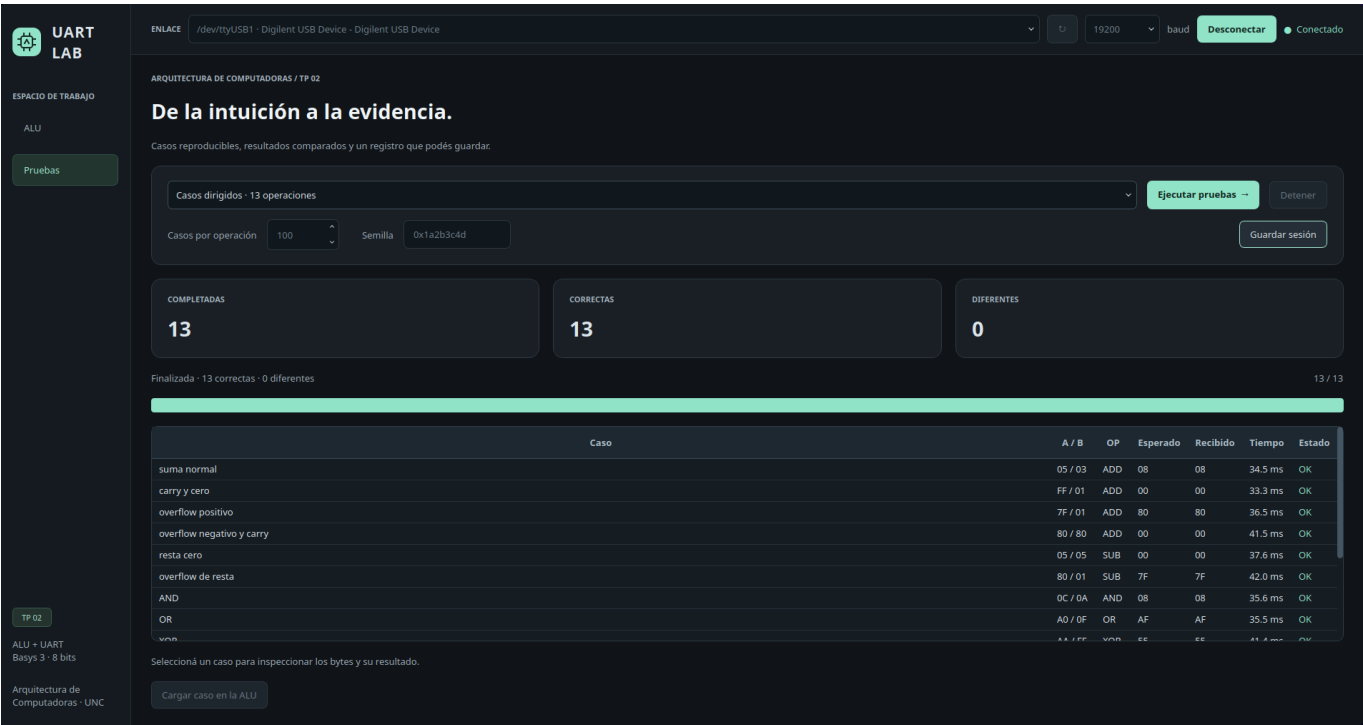


Figura 9.3: Resultado de la suite dirigida ejecutada sobre la Basys 3: 13 de 13 casos correctos.

9.3.2 Casos pseudoaleatorios

La segunda ejecución utilizó cien pares de operandos por cada una de las ocho operaciones. La semilla `0x1a2b3c4d` fijó la secuencia y permitió que el conjunto de 800 transacciones fuera reproducible. La prueba terminó con 800 respuestas correctas y ninguna diferencia respecto del modelo, según la figura 9.4.

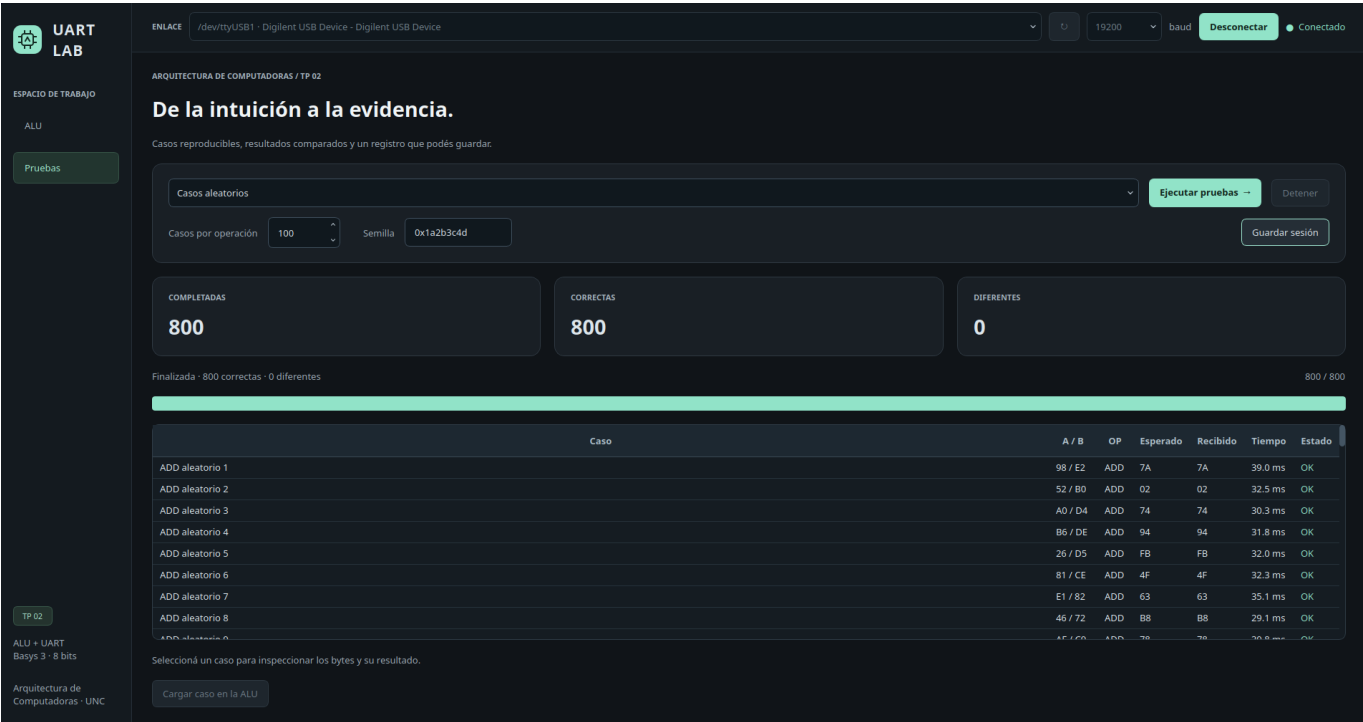


Figura 9.4: Resultado de la suite pseudoaleatoria: 800 de 800 respuestas coincidentes con el modelo.

9.3.3 Correspondencia entre UART y LED

La respuesta UART contiene únicamente el resultado de ocho bits. Las banderas Z, C y V que presenta la GUI son los valores calculados localmente por el cliente python y no datos recibidos desde la FPGA. Por esa razón se repitieron manualmente dos casos de frontera y se comparó la respuesta de la GUI con los LED físicos.

La operación `FF + 01` produjo la respuesta UART `00`. En la GUI, el byte recibido coincidió con el esperado; en la placa quedaron encendidos LD8 (Z) y LD9 (C), mientras que LD10 (V) y los LD0–LD7 permanecieron apagados. El patrón completo fue, por lo tanto, `0x300`. La figura 9.5 reúne ambas observaciones.

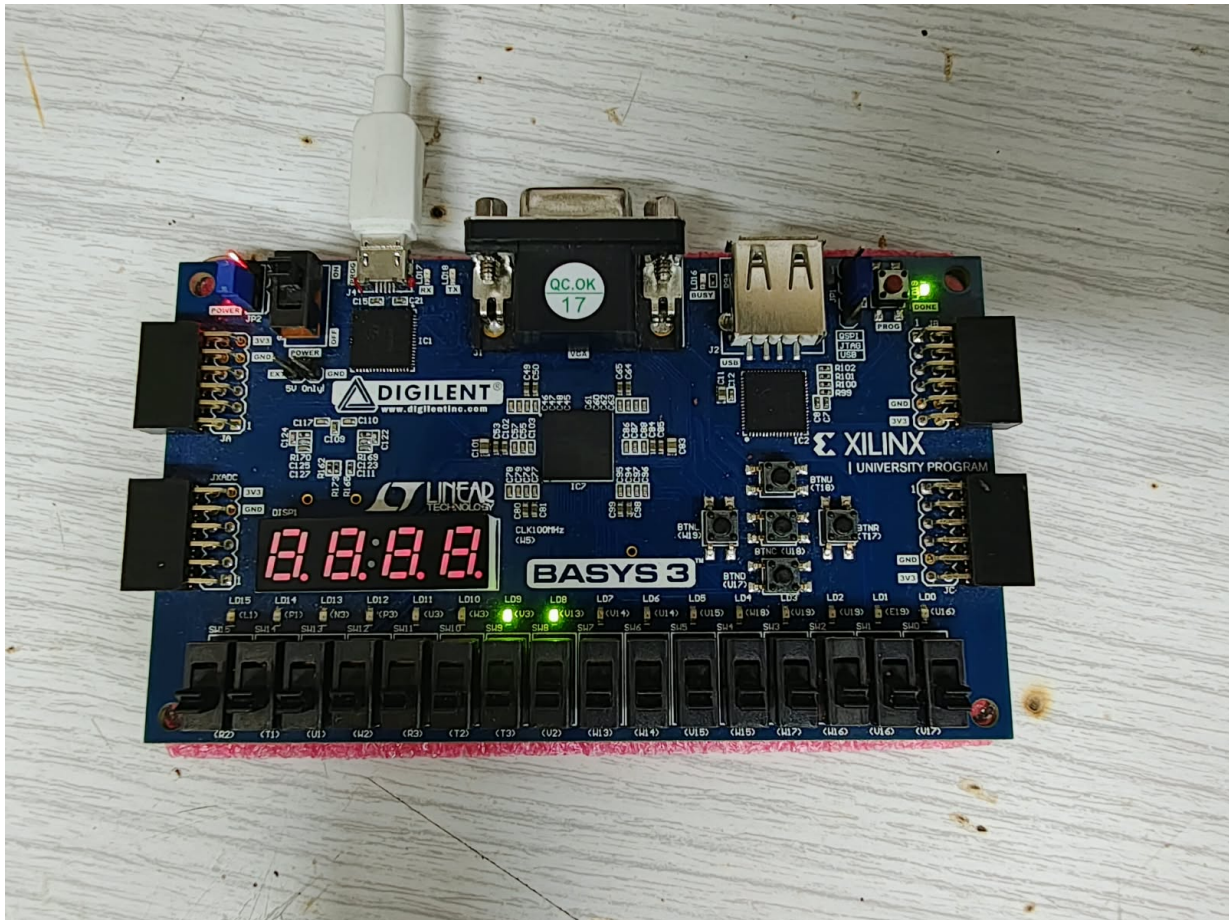
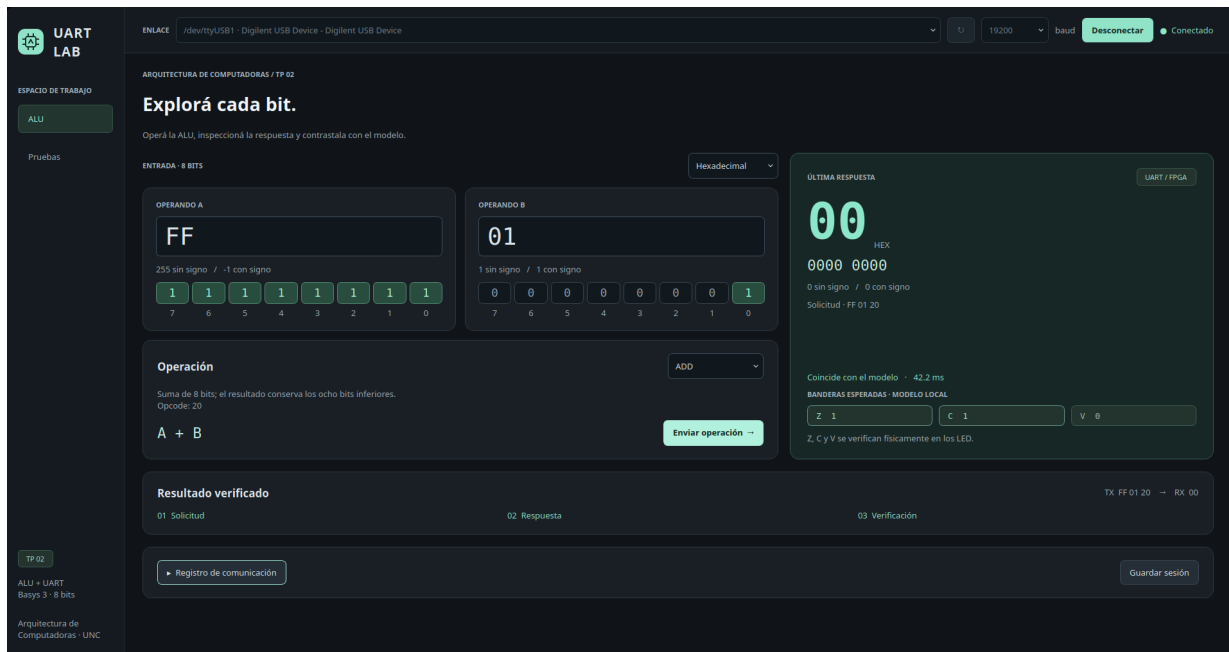


Figura 9.5: Suma $FF + 01$: respuesta 00 en la GUI y banderas Z y C activas en la placa.

La operación $7F + 01$ produjo la respuesta UART 80 . El bit 7 del resultado se reflejó en el LD7 y el overflow en el LD10; los demás LED permanecieron apagados. La palabra observada fue $0x480$, en correspondencia con el resultado y las banderas calculadas por el modelo. La figura 9.6 muestra la comprobación.

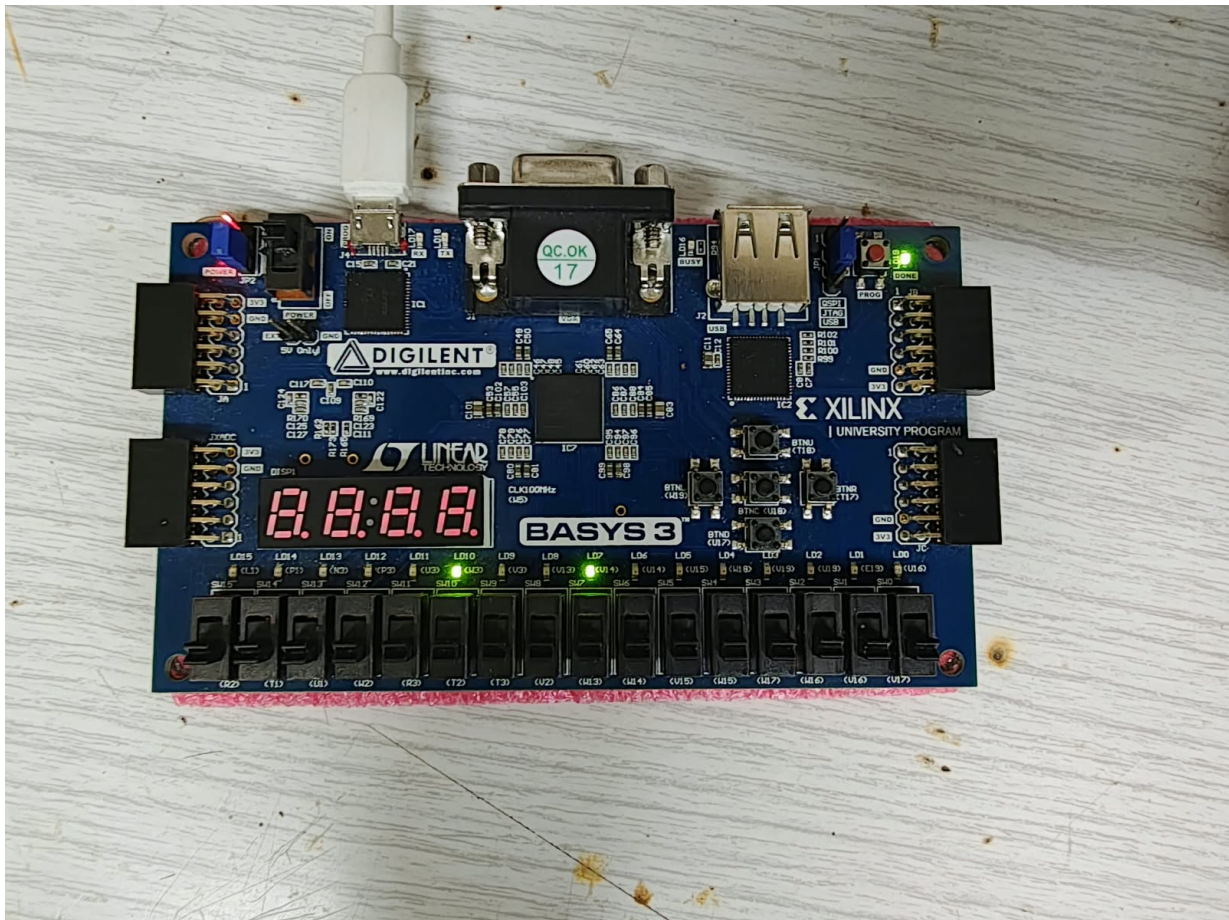
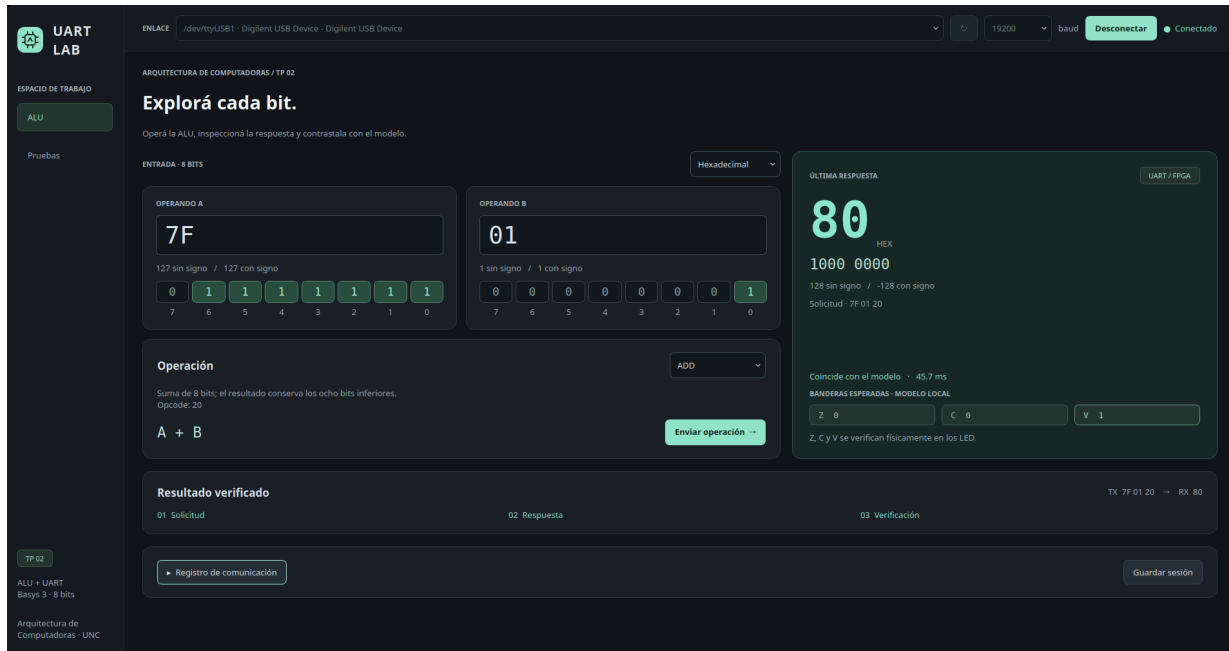


Figura 9.6: Suma $7F + 01$: respuesta 80 en la GUI y overflow activo en la placa.

9.4 Resultados y criterio de aceptación

Antes de cada ensayo se fijó el resultado esperado. Se consideró aprobada una suite cuando todas las respuestas coincidieron con el modelo, y una prueba visual cuando el resultado UART y el patrón de LED representaron la misma operación. La tabla 9.1 resume las observaciones.

Prueba	Criterio	Resultado observado	Estado
Programación	Dispositivo programado y DONE activo.	<code>xc7a35t_0</code> en estado <i>Programmed</i> .	Aprobada
Conexión UART	Enlace 8N1 establecido a 19 200 baud.	<code>/dev/ttyUSB1</code> ; GUI en estado <i>Conectado</i> .	Aprobada
Suite dirigida	13 operaciones sin diferencias.	13 correctas; 0 diferentes.	Aprobada
Suite aleatoria	800 operaciones sin diferencias.	800 correctas; 0 diferentes.	Aprobada
ADD FF+01	UART 00 ; LED = 0x300 .	Respuesta 00 ; LD8 y LD9 encendidos.	Aprobada
ADD 7F+01	UART 80 ; LED = 0x480 .	Respuesta 80 ; LD7 y LD10 encendidos.	Aprobada

Tabla 9.1: Resultados de la validación física.

La GUI transmite solicitudes completas de tres bytes y no ofrece una acción para generar deliberadamente una trama parcial. En consecuencia, la recuperación por timeout no formó parte de esta campaña física. Ese mecanismo se verificó de manera específica en los bancos de prueba del controlador y del sistema integrado, donde sí fue posible controlar con precisión el intervalo entre bytes.

9.5 Interpretación

Las pruebas físicas abarcaron la cadena completa: selección del puerto, transmisión desde la GUI, recepción UART, almacenamiento temporal, interpretación de los dos operandos y el opcode, ejecución de la ALU, transmisión de la respuesta y retención del resultado en los LEDs. La suite dirigida ejercitó situaciones de frontera y la suite pseudoaleatoria amplió el conjunto con 800 operaciones reproducibles. En total se completaron 815 transacciones físicas sin diferencias respecto del modelo de referencia calculado localmente.

Las comprobaciones visuales agregaron evidencia que el byte UART no puede aportar por sí solo: demostraron que el resultado retenido y las banderas Z, C y V correspondieron a la misma operación informada a la computadora. En conjunto, los resultados de la tabla 9.1 permiten dar por aceptado el funcionamiento extremo a extremo del sistema en la Basys 3.

Sección 10

Conclusiones

Como se planteó en la sección 1, este trabajo cambió el centro del problema respecto del TP1. La ALU siguió siendo el bloque encargado de realizar las operaciones, pero ya no recibió los operandos de forma directa mediante los switches de la placa. En este caso fue necesario construir una UART capaz de recibirlos y transmitir el resultado de manera serial. Esto hizo que el diseño dejara de depender solamente de los valores presentes en las entradas y pasara a depender también del orden en que llegan y del momento en que se producen los distintos eventos.

Como se vio en la sección 4, dedicada al núcleo UART, las máquinas de estado finitas permitieron organizar este comportamiento temporal de una forma clara. En la recepción, la FSM espera el inicio de una trama, valida el bit de start, reconstruye el byte a medida que llegan sus bits y comprueba finalmente el bit de stop. En la transmisión ocurre el recorrido inverso: a partir de un byte se generan, en el orden y durante el tiempo correspondiente, el start, los ocho bits de datos y el stop. El generador de ticks y el sobremuestreo proporcionan la referencia temporal para que ambas máquinas puedan avanzar por esas etapas y trabajar con una señal que no comparte el reloj de la FPGA.

La tercera máquina de estados, presentada en la sección 4.9, unió la comunicación con la ALU. Sus estados representan las distintas etapas de una operación: esperar los dos operandos, recibir el código de operación, registrar el resultado y entregarlo para su transmisión. Esta separación permitió expresar de manera directa qué información debía conservarse en cada momento y evitó que la interpretación del protocolo quedara mezclada con la recepción bit a bit. De esta forma, las tres FSM resolvieron partes diferentes del mismo intercambio y, al trabajar en conjunto, hicieron posible completar el recorrido desde la entrada RX hasta la respuesta por TX.

La implementación también permitió llevar a la práctica los criterios de diseño de máquinas de estado estudiados en la consigna. La distinción entre el estado actual, el próximo estado y las salidas hizo más sencillo seguir las transiciones tanto en el código como en las formas de onda. A su vez, la codificación one-hot resultó adecuada para la FPGA y facilitó reconocer cada estado durante la verificación. Las ramas de recuperación y los estados de espera fueron igualmente importantes, porque una UART no solo debe procesar tramas válidas, sino también volver a una situación conocida frente a una entrada incompleta, un error de trama o un estado no contemplado.

Además de lo requerido para implementar la UART, se incorporaron recursos que mejoraron la robustez del sistema. Los sincronizadores acondicionaron las señales externas, mientras que las colas FIFO permitieron desacoplar por un breve intervalo la comunicación y el procesamiento. El timeout y el manejo de esperas completaron esa idea al impedir que una operación incompleta o una salida momentáneamente ocupada dejara al controlador en una situación indefinida. Si bien estos elementos no constituyen el objetivo principal del trabajo, ayudaron a integrar las máquinas de estado en un sistema más confiable y cercano a una aplicación real.

Finalmente, las pruebas presentadas en las secciones 6 y 9 permitieron observar las transiciones de cada FSM por separado y comprobar después su funcionamiento conjunto. Las simulaciones mostraron la recepción y transmisión de tramas, la secuencia seguida por el controlador y la recuperación ante errores, mientras que los ensayos sobre la Basys 3 confirmaron que una solicitud enviada desde la computadora podía atravesar todo el sistema y producir la respuesta esperada. En conjunto, el trabajo permitió comprender que una UART no se reduce a desplazar bits: requiere modelar una secuencia completa de acciones, conservar el progreso entre ciclos y definir con precisión cómo avanzar o recuperarse. En ese sentido, las máquinas de estado fueron la herramienta central que permitió transformar el protocolo UART en un circuito digital ordenado, verificable y sintetizable.

Referencias

- Cátedra de Arquitectura de Computadoras, *Máquinas de Estado Finitas — TP: UART*, filminas y consigna provistas en [5 - Maquinas de Estado Finitas.pdf](#).
- Arnaudo, F. A. y Perotti, F. J., *Unidad Aritmético-Lógica*, Trabajo Práctico N° 1, fuentes SystemVerilog, testbenches e informe incluidos en este repositorio.
- Pong P. Chu, *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*, capítulo sobre comunicación serie UART, Wiley, 2008.
- Digilent, *Basys 3 Reference Manual*. Disponible en https://digilent.com/reference/_media/reference/programmable-logic/basys-3/basys3_rm.pdf.
- Digilent, *Basys 3 Master XDC*. Disponible en <https://github.com/Digilent/digilent-xdc>.
- AMD, *Vivado Design Suite User Guide: Programming and Debugging*, UG908, procedimiento de conexión y programación mediante Hardware Manager. Disponible en <https://docs.amd.com/r/2025.1-English/ug908-vivado-programming-debugging/Opening-Hardware-Target-Connections>.
- AMD, *Vivado Design Suite User Guide: Using Constraints*, UG903, documentación de restricciones y análisis temporal.
- IEEE, *Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE 1800.